SOCKET BASICS

Byeong-hee Roh
Dept. of Software and Computer Engineering
Ajou University

변화와 융합의 중심
MMcN
멀티미디어 융합 연구실
Mobile Multimedia convergecne Network Lab.
http://mmcn.ajou.ac.kr

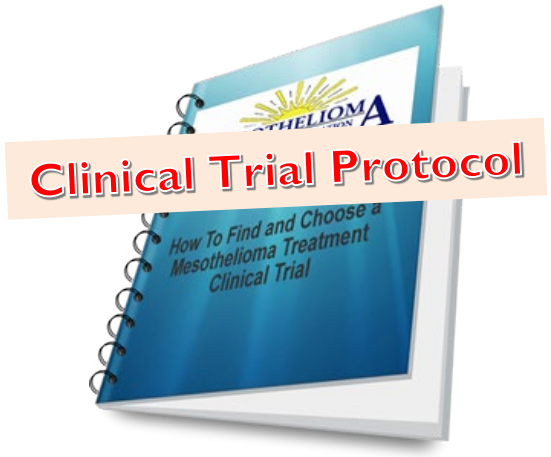
Contents

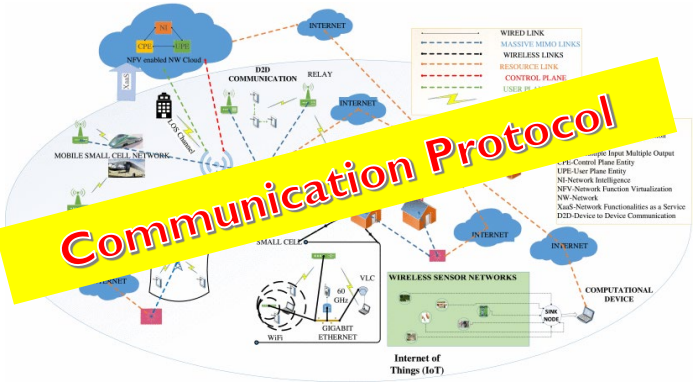
- Definition of Protocol
- TCP/IP Layered Model
- Socket Basics
- Socket Functions
- Socket Address
- IP Address Manipulation
- Binding

Definition of Protocol

Definition of Protocol

- Society
  - etiquette
  - (politics), a formal agreement between nation states
- Science
  - (science), a predefined written procedural method of conducting experiments
  - (clinical trial), a document that plans a clinical trial
- Communications
  - a defined set of rules and regulations that determine how data is transmitted
  - in telecommunications and computer networking

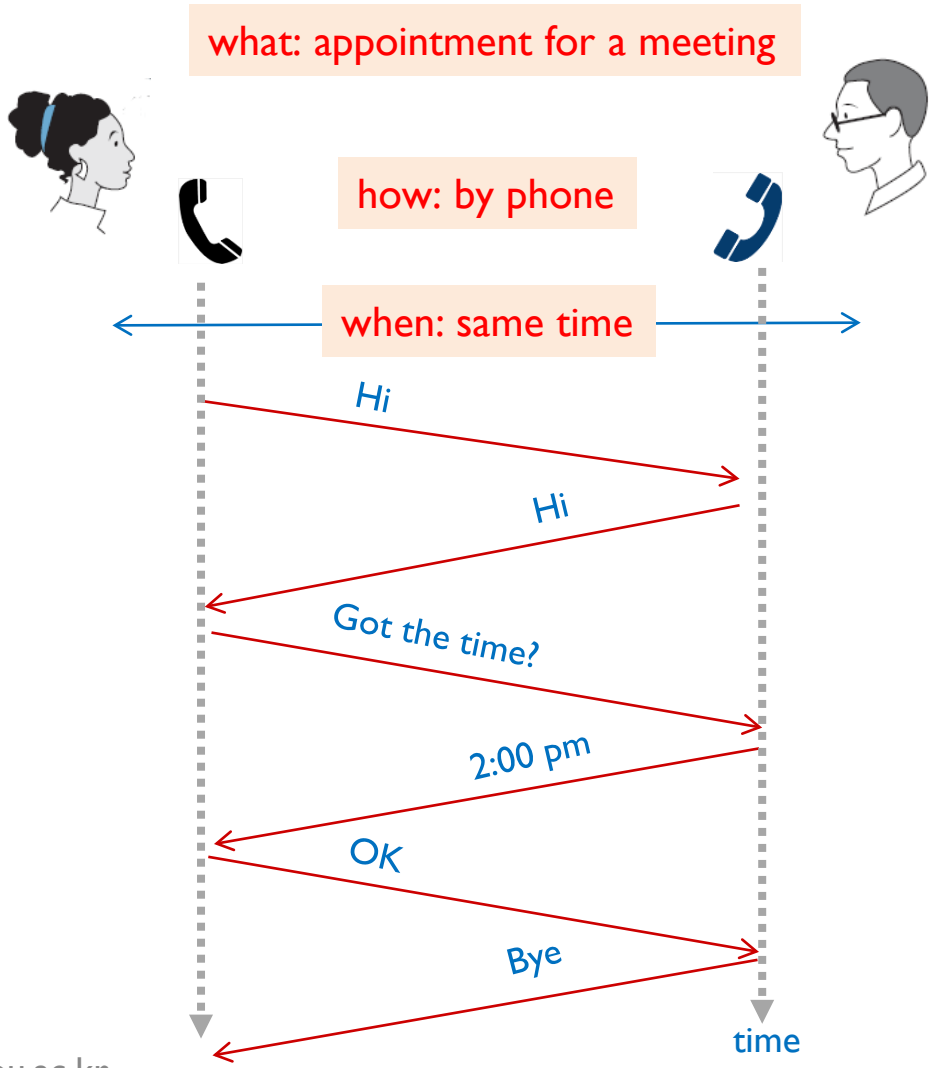




Definition of (Communication) Protocol

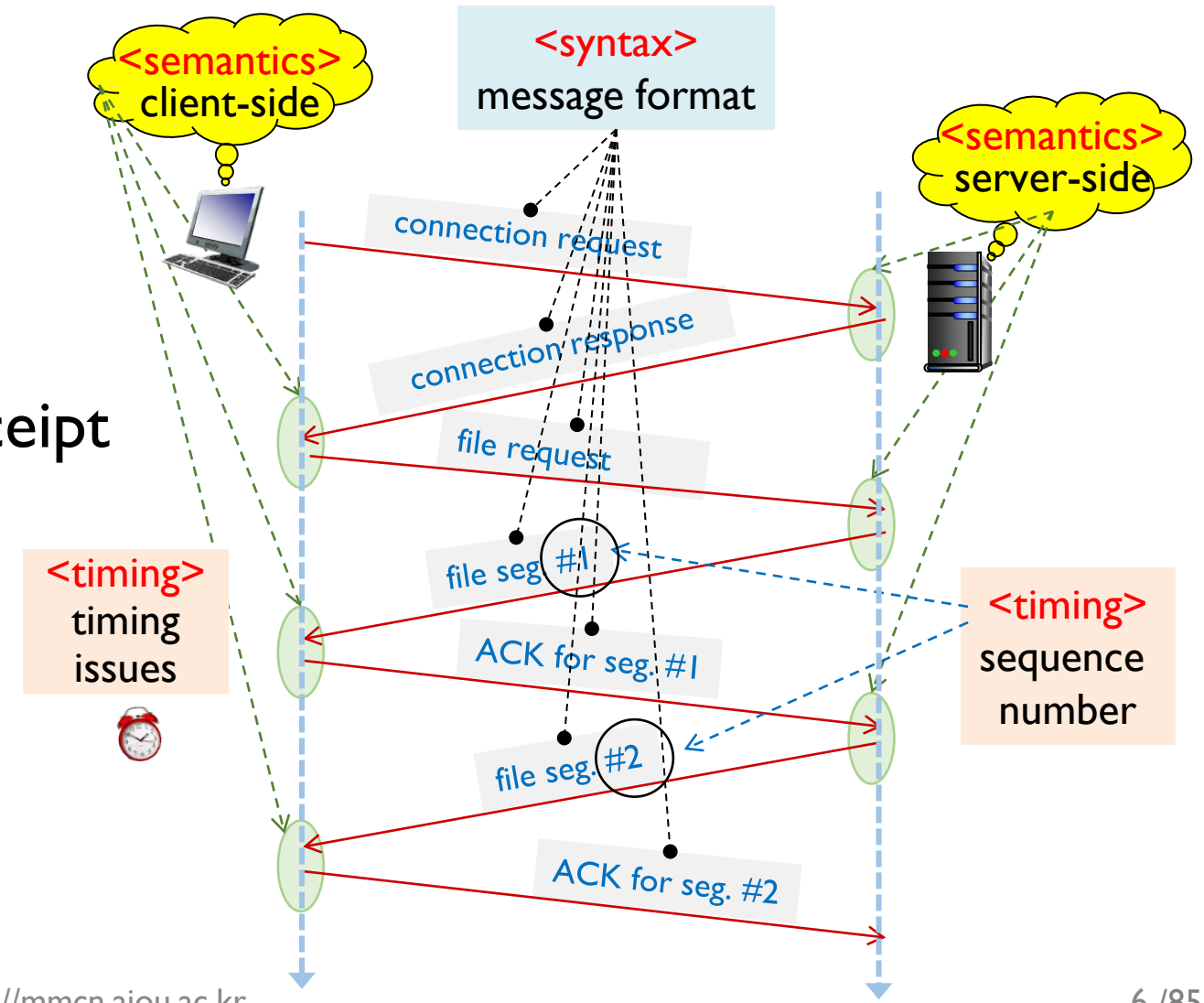
- For two communicating entities to successfully communicate,
  - what is communicate,
  - how it is communicated, and
  - when it is communicated

  must conform to some mutually acceptable set of conventions between the entities involved
- Definition of Protocol
  - the set of conventions for the communicating entities to exchange information (messages) successfully



Key Elements of a Protocol

- **Syntax** : data format
  - data formats
  - encoding/decoding information
  - signal level, …
- **Semantics** : actions taken on message transmission and receipt
  - response
  - control information
  - error handling, …
- **Timing** : order of messages
  - Timeout
  - sequence number
  - speed matching, …

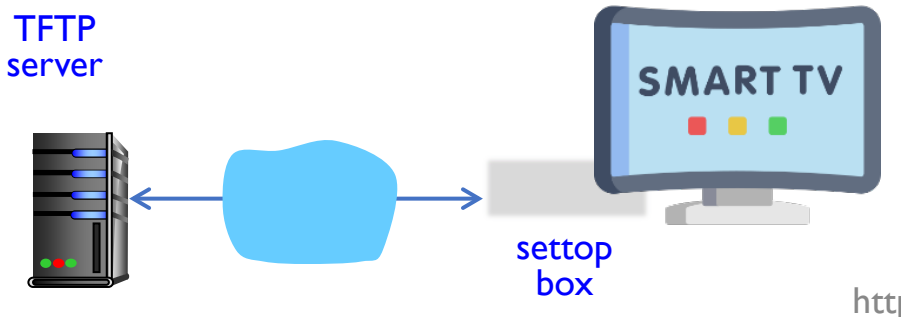


Key Elements of a Protocol

- Network Programming is the task for implementing target applications associated with network by designing
  - syntax
  - semantics
  - timing
  - and others if necessary

Key Elements of a Protocol

- Example :TFTP (1/2)
  - Trivial File Transfer Protocol
    - a simple lockstep File Transfer Protocol
    - a client can get (or put) a file from (or onto) a remote host
    - usage example
      - transfer firmware images and configuration files to network appliances
      - e.g. routers, settop boxes, smart TVs, and so on



2 octets	variable	1 octet	variable	1 octet
Read request (1)	File name	0	Mode	0

(a) Type 1

2 octets	variable	1 octet	variable	1 octet
write request (2)	File name	0	Mode	0

(b) Type 2

2 octets	2 octets	upto 512 octets
Data (3)	Sequence number	Data

(C) Type 3

2 octets	2 octets
ACK (4)	Sequence number

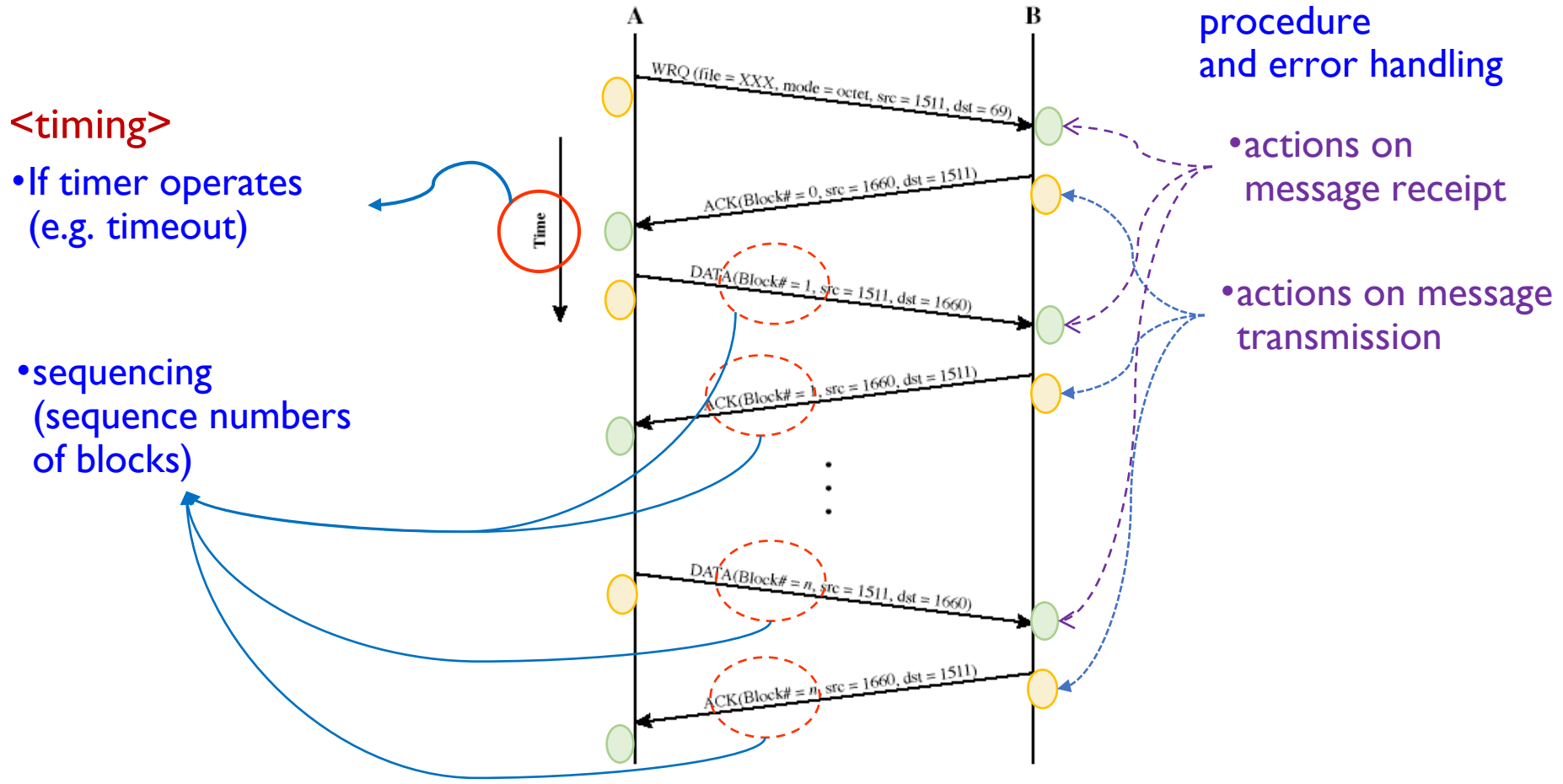
(d) Type 4

Type of FTFP Messages

<syntax> :TFTP packet formats

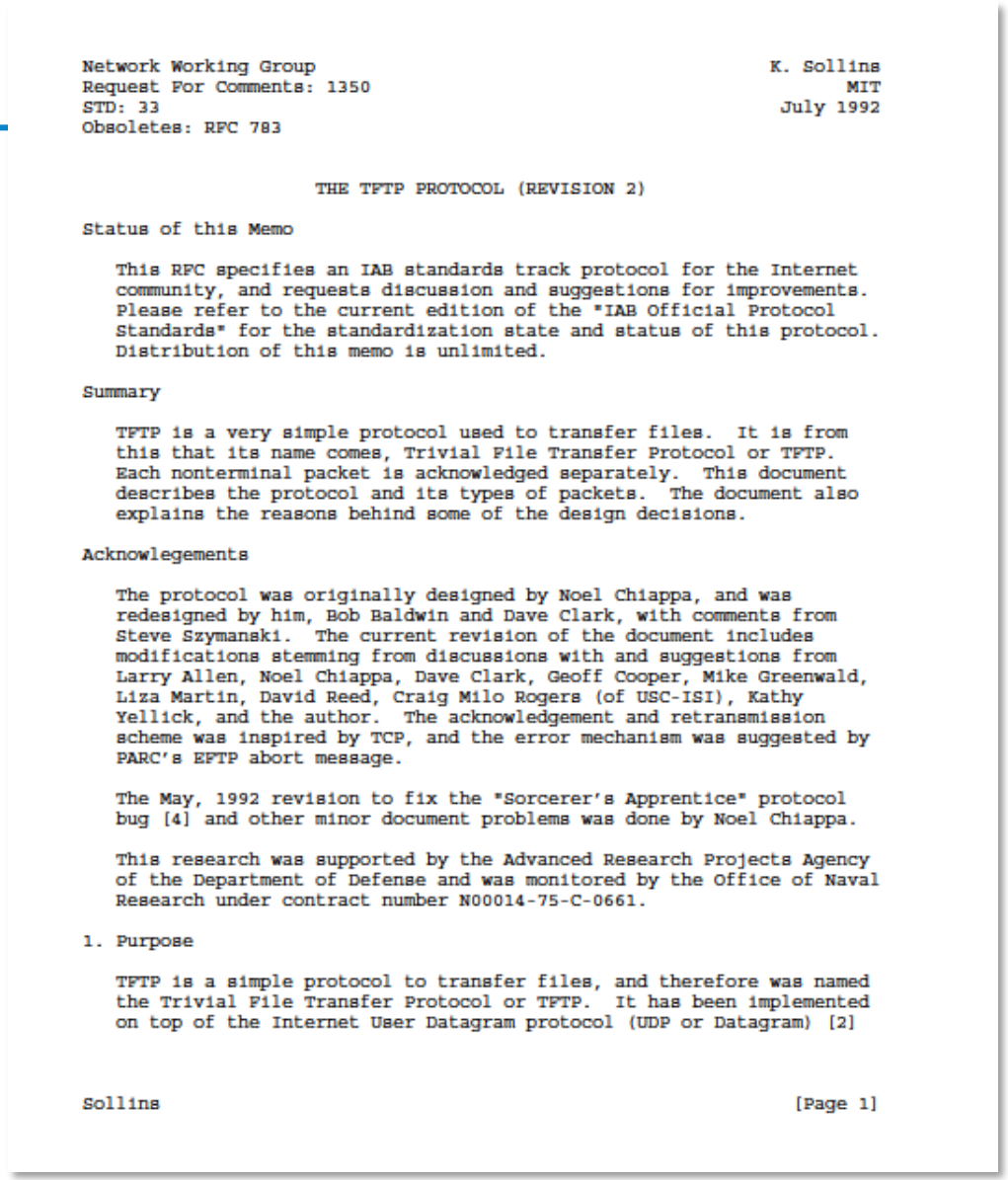
Key Elements of a Protocol

- Example :TFTP (2/2)
  - Trivial File Transfer Protocol



Key Elements of a Protocol

- RFC 1350: The TFTP Protocol (Revision 2)
  - Purpose
  - Protocol Overview
  - Syntax
    - 5. TFTP Packets
    - I. Appendix
  - Semantics
    - 4. Initial Connection Protocol
    - 6. Normal Termination
    - 7. Premature Termination
  - References
  - Security Considerations



Network Working Group
Request For Comments: 1350
STD: 33
Obsoletes: RFC 783

K. Sollins
MIT
July 1992

THE TFTP PROTOCOL (REVISION 2)

Status of this Memo

This RFC specifies an IAB standards track protocol for the Internet community, and requests discussion and suggestions for improvements. Please refer to the current edition of the "IAB Official Protocol Standards" for the standardization state and status of this protocol. Distribution of this memo is unlimited.

Summary

TFTP is a very simple protocol used to transfer files. It is from this that its name comes, Trivial File Transfer Protocol or TFTP. Each nonterminal packet is acknowledged separately. This document describes the protocol and its types of packets. The document also explains the reasons behind some of the design decisions.

Acknowlegements

The protocol was originally designed by Noel Chiappa, and was redesigned by him, Bob Baldwin and Dave Clark, with comments from Steve Szymanski. The current revision of the document includes modifications stemming from discussions with and suggestions from Larry Allen, Noel Chiappa, Dave Clark, Geoff Cooper, Mike Greenwald, Liza Martin, David Reed, Craig Milo Rogers (of USC-ISI), Kathy Yellick, and the author. The acknowledgement and retransmission scheme was inspired by TCP, and the error mechanism was suggested by PARC's EFTP abort message.

The May, 1992 revision to fix the "Sorcerer's Apprentice" protocol bug [4] and other minor document problems was done by Noel Chiappa.

This research was supported by the Advanced Research Projects Agency of the Department of Defense and was monitored by the Office of Naval Research under contract number N00014-75-C-0661.

1. Purpose

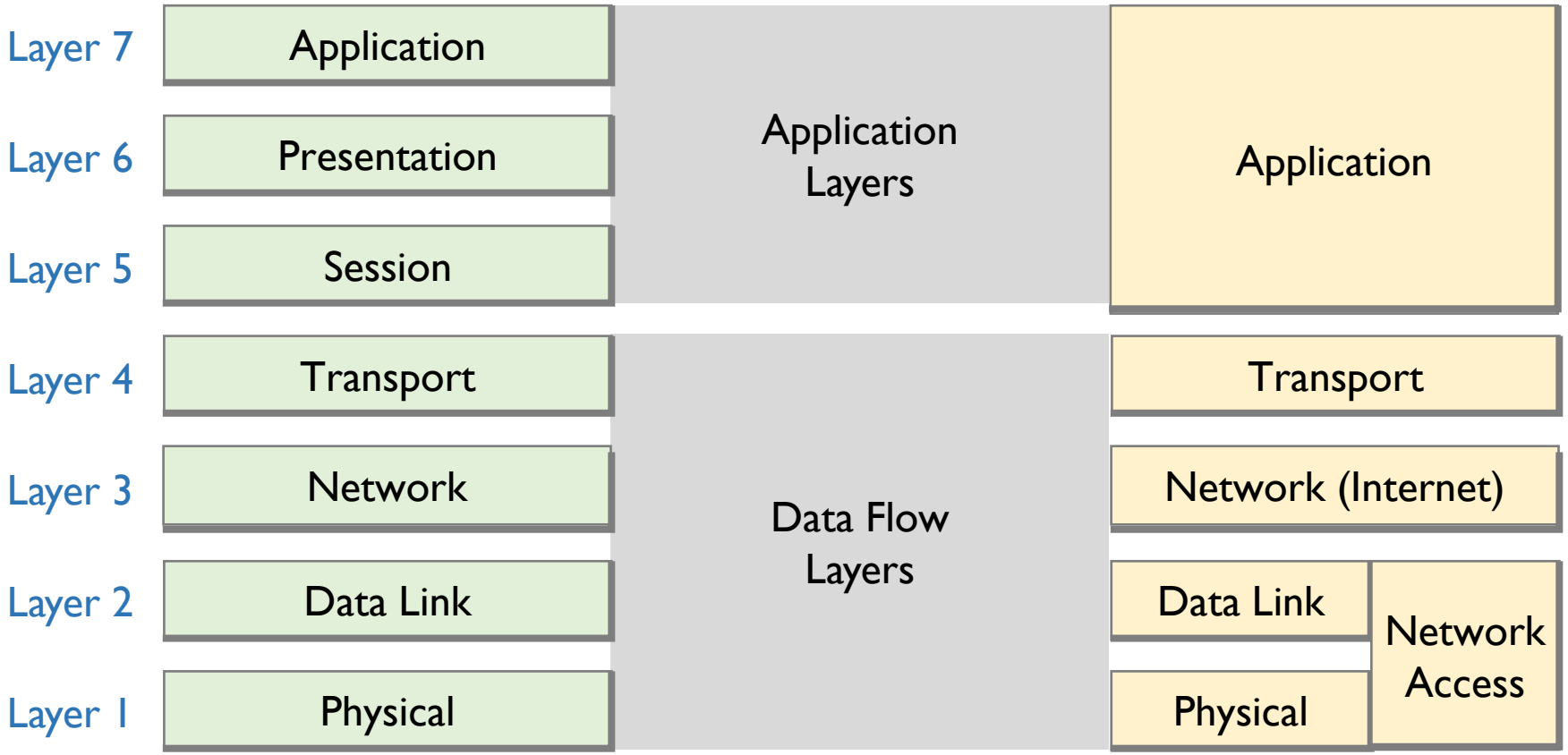
TFTP is a simple protocol to transfer files, and therefore was named the Trivial File Transfer Protocol or TFTP. It has been implemented on top of the Internet User Datagram protocol (UDP or Datagram) [2]

Sollins

[Page 1]

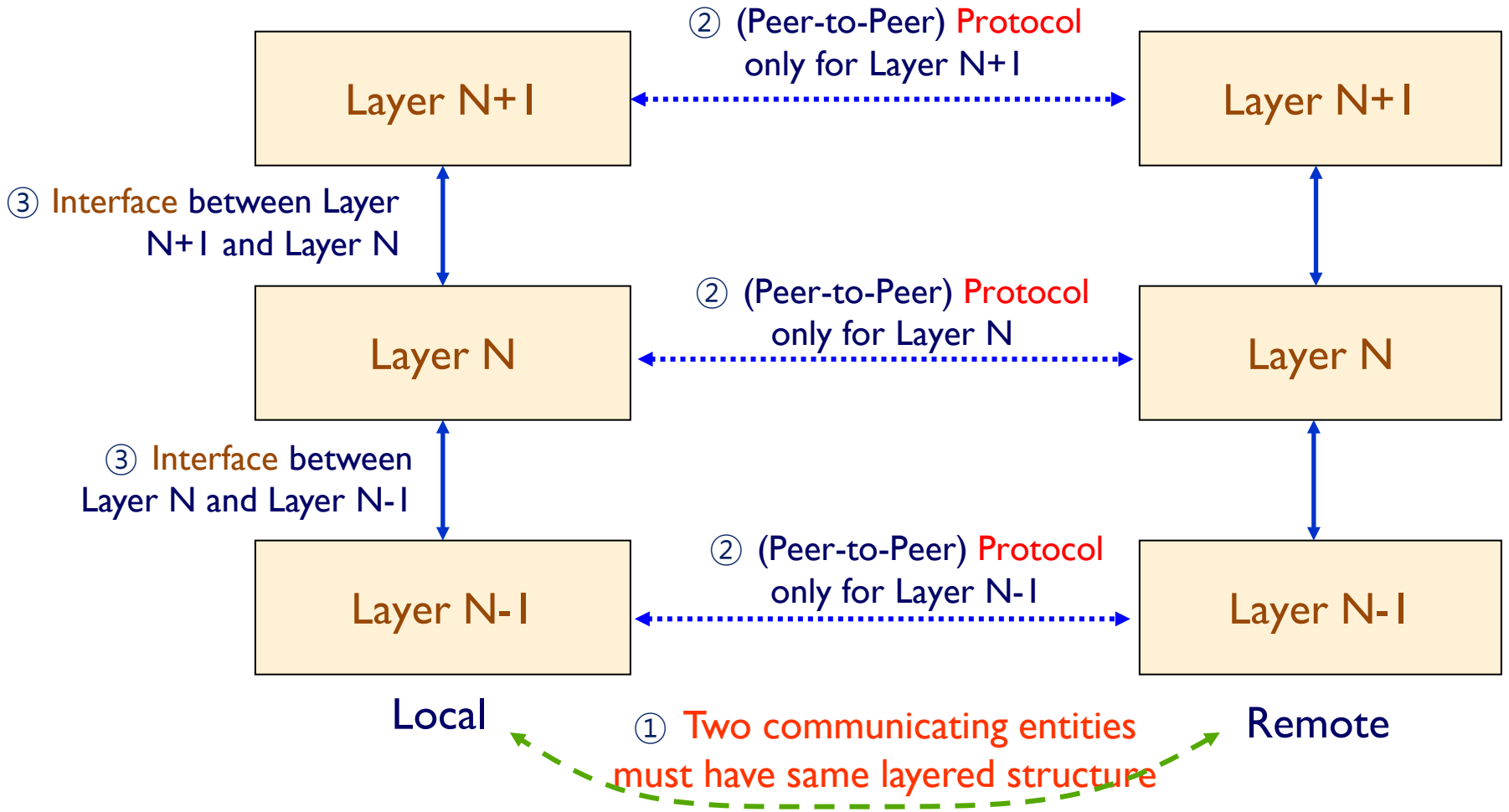
Layered Protocol Reference Model

❖ OSI and TCP/IP Layered Models



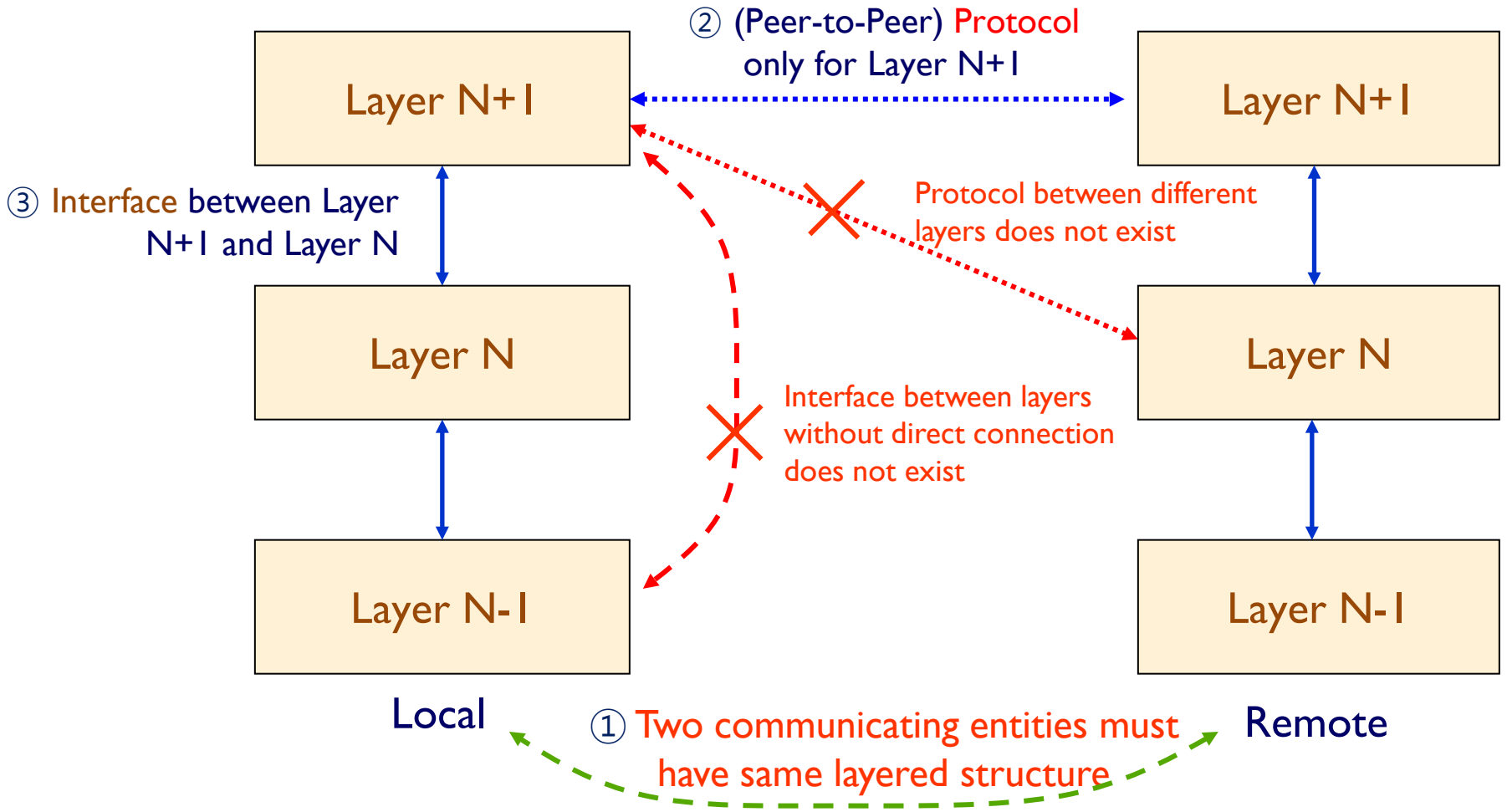
Layered Protocol Reference Model

❖ Protocols and Interfaces



Layered Protocol Reference Model

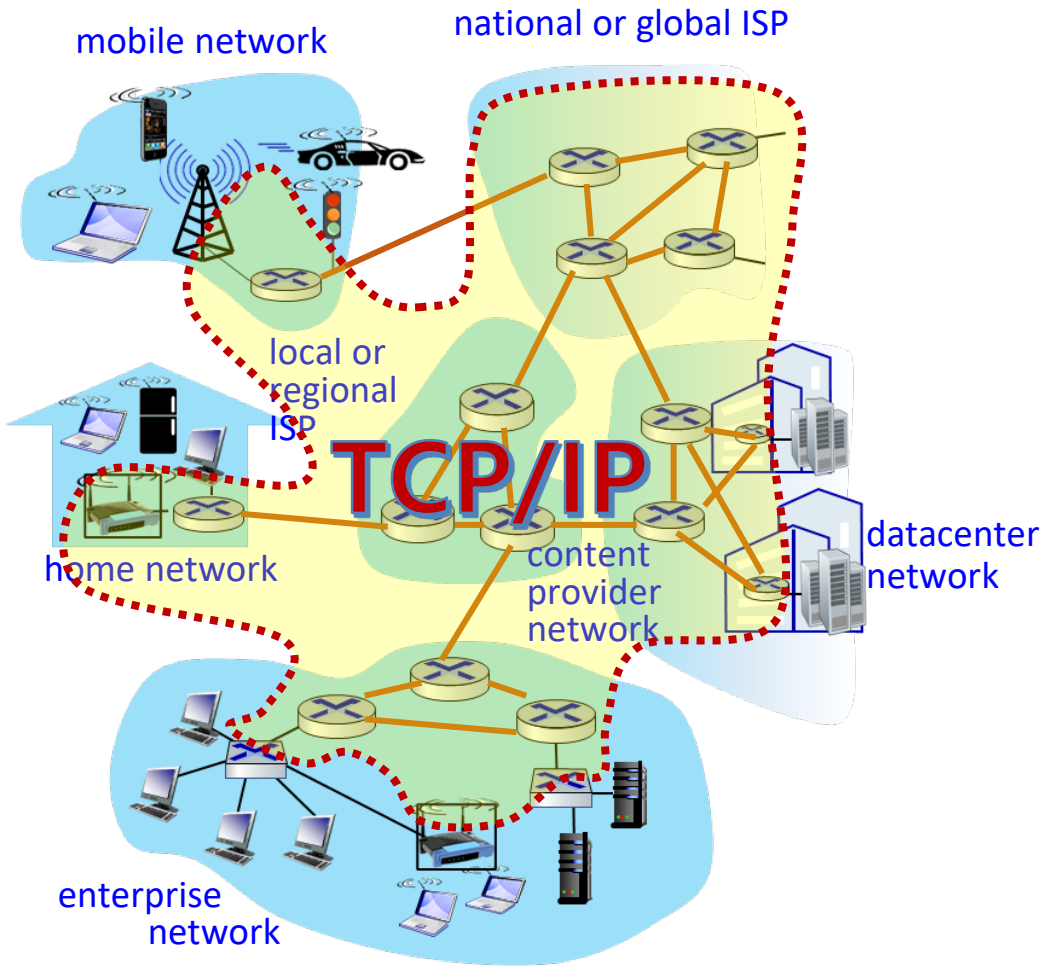
❖ Independence of Protocol and Interface



TCP/IP Layered Model

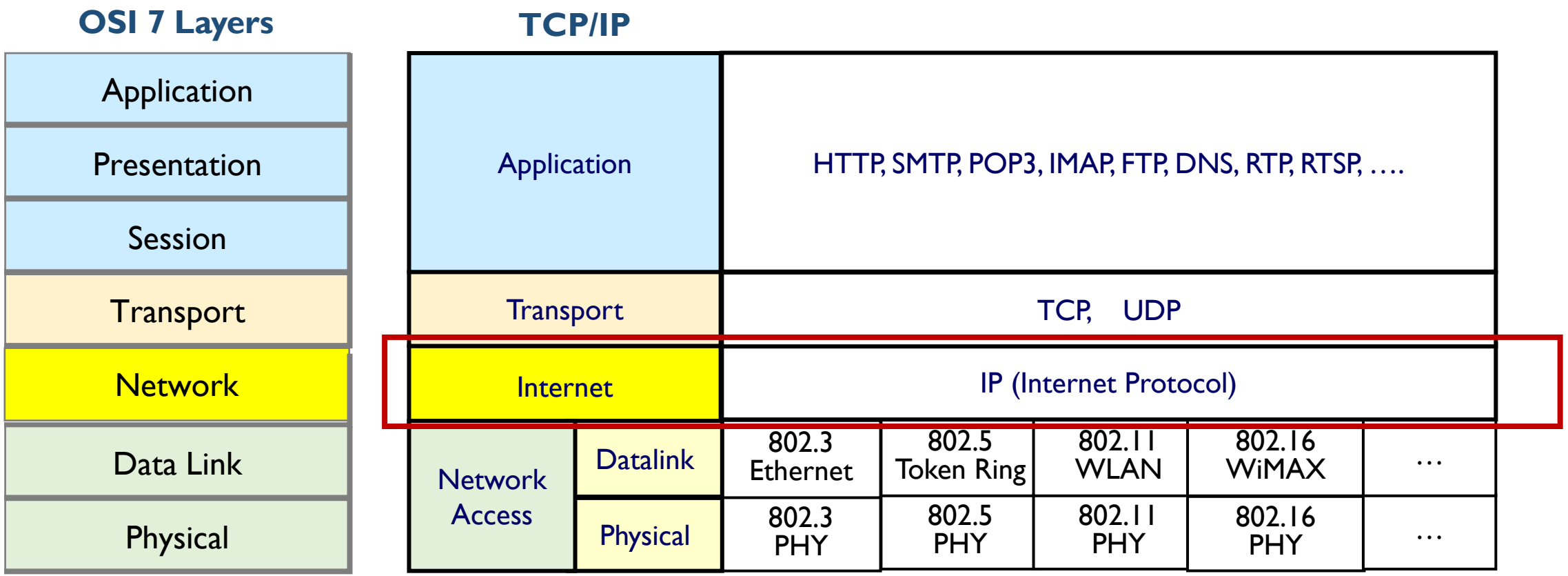
"the Internet"

- the Internet (as a proper noun)
  - network of networks
    - network of globally interconnected computer networks
  - based on
    - the standard Internet protocol suite
    - TCP/IP
  - It consists of millions of
    - connected computing devices,
    - communication links
    - network devices
    - people
    - and everything



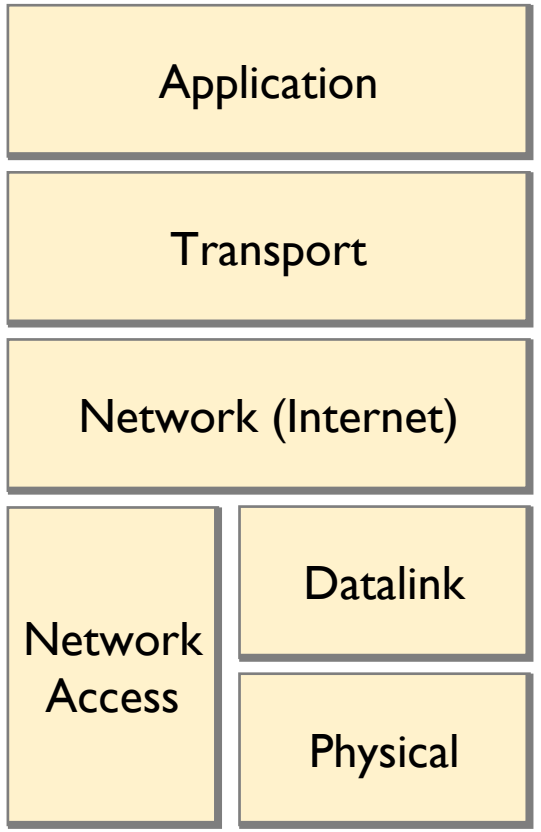
TCP/IP Protocol Model

- IP – the only one protocol at network layer of TCP/IP model
  - many protocols exist at other layers (phy, datalink, transport, application)



TCP/IP Protocol Model

- Five layers
  - Application
    - supporting network applications
      - FTP, SMTP, HTTP, e-mail, P2P, …
  - Transport
    - data transfer between app. processes
      - TCP (Transmission Control Protocol)
      - UDP (User Datagram Protocol)
  - Network (IP)
    - routing of datagrams from source to destination
      - IP (Internet Protocol)
      - optionally, ICMP, IGMP, ARP, …
  - Link
    - frame delivery between neighboring network elements
      - Ethernet, 802.11 (WiFi), …
  - Physical
    - bits "on the wire"

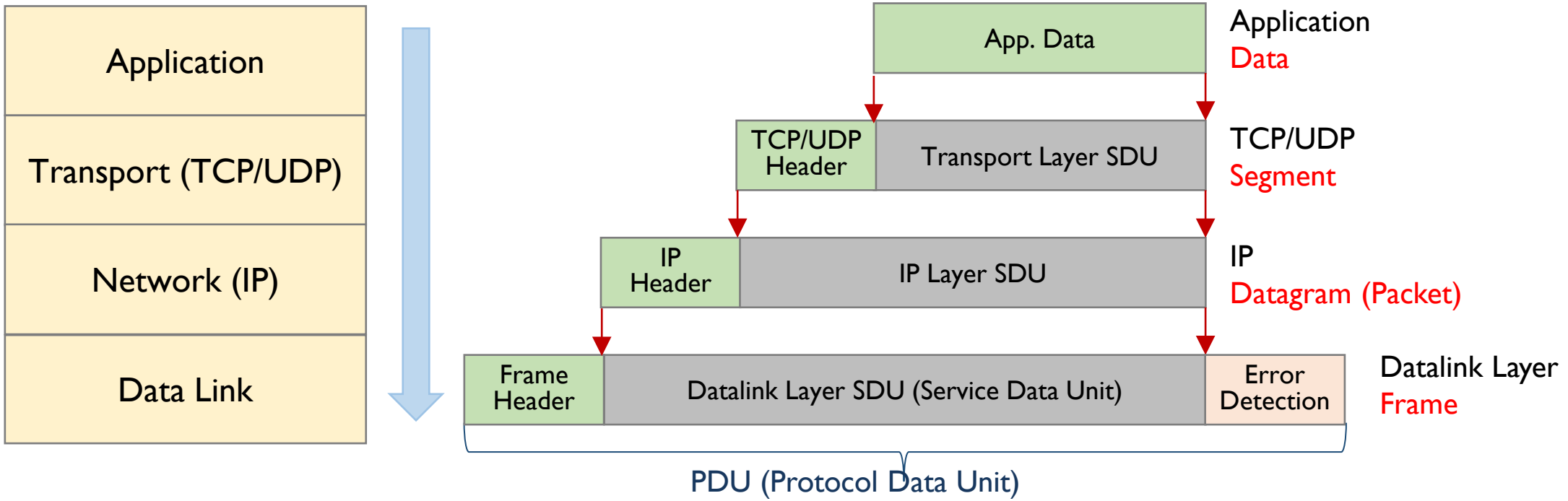


TCP/IP Model

TCP/IP – Encapsulation

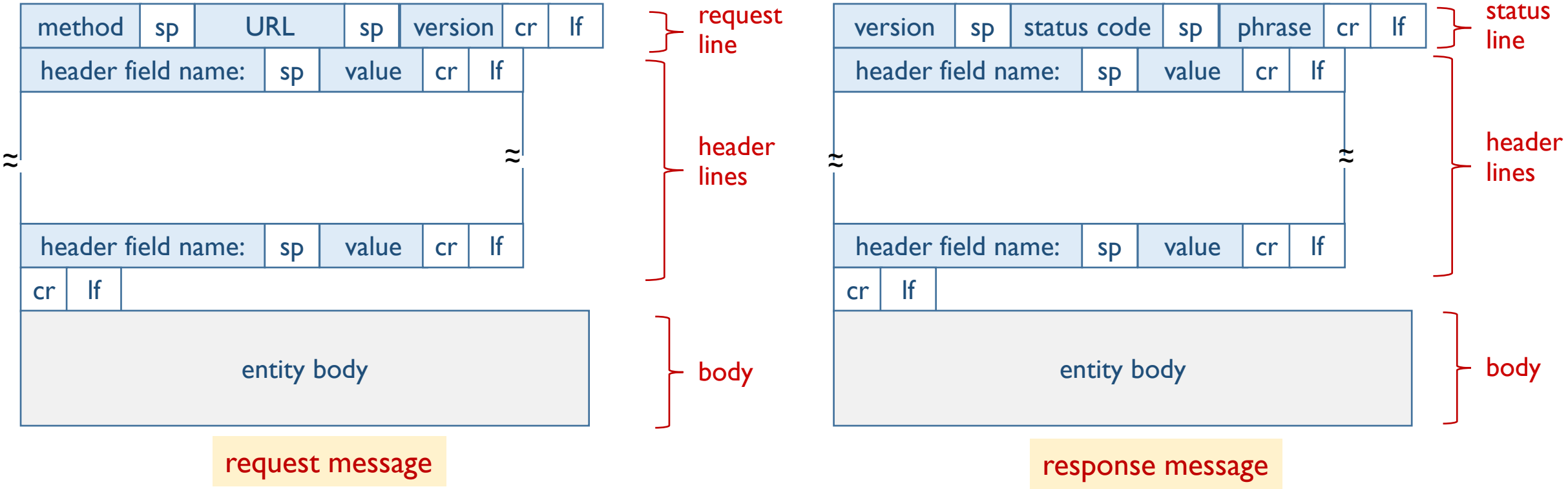
- Interfacing towards lower layers
  - each layer
    - takes a message (SDU) from its upper layer, but
    - does not modify any bit of the message, and
    - appends additional information (header)
    - delivers PDU (header+SDU) to its lower layer

SDU (Service Data Unit)
PDU (Protocol Data Unit)



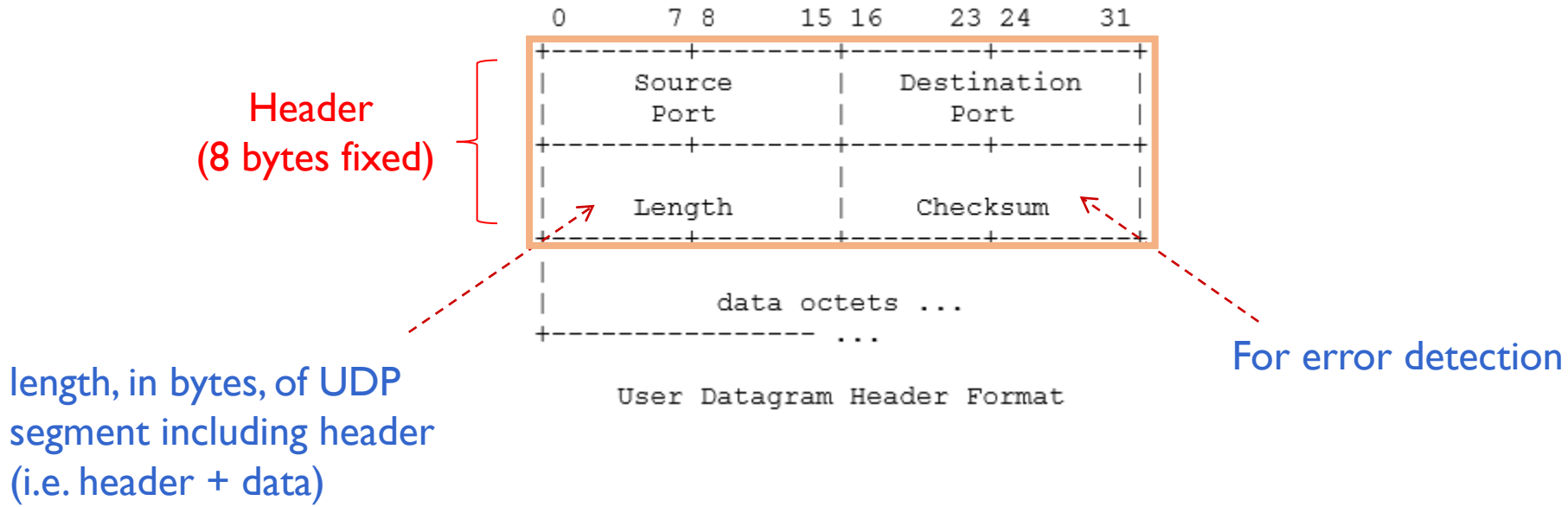
Application Layer: HTTP Message Format

- Two types of HTTP messages
  - request
  - response



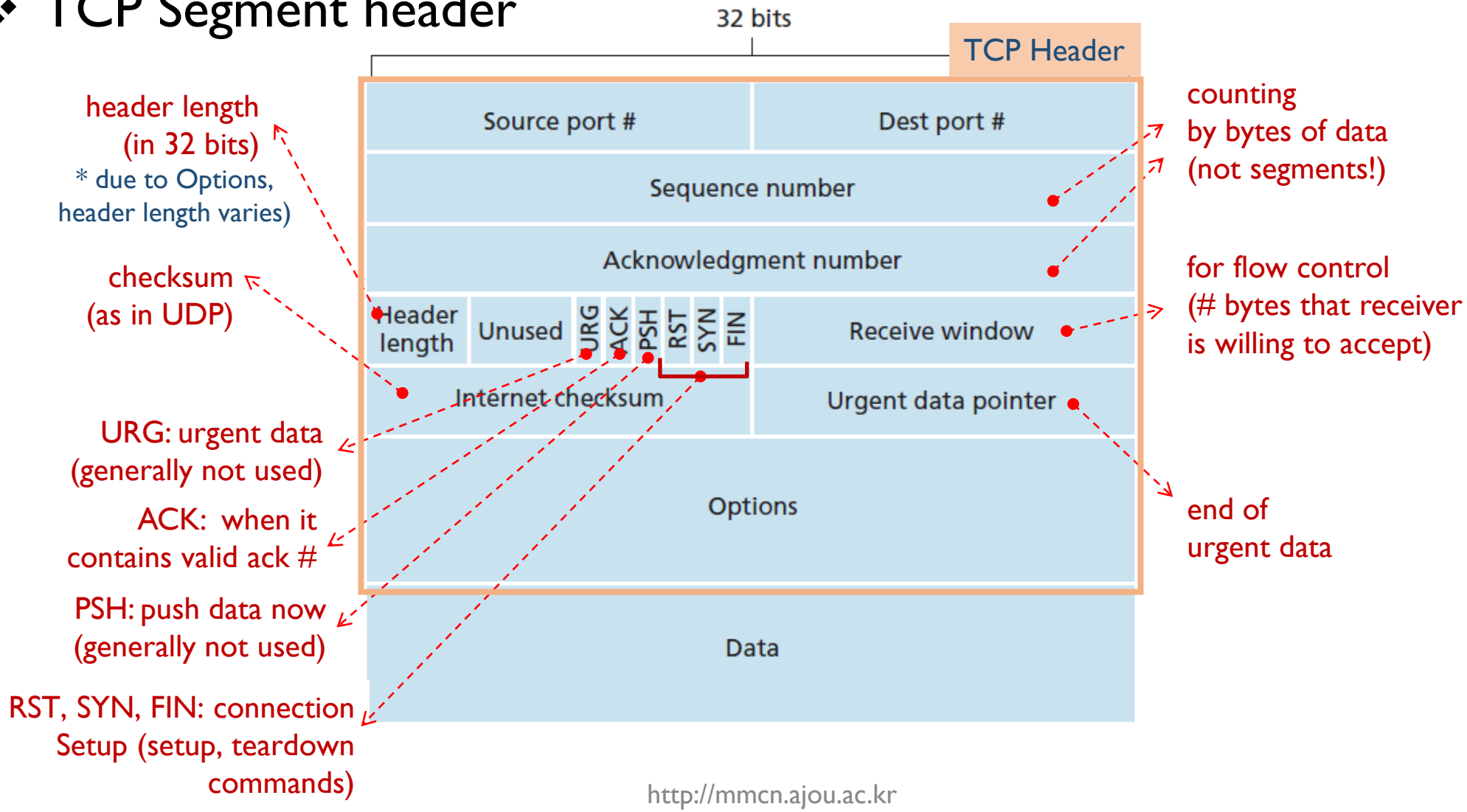
Transport Layer Protocols

❖ UDP Segment header



Transport Layer Protocols

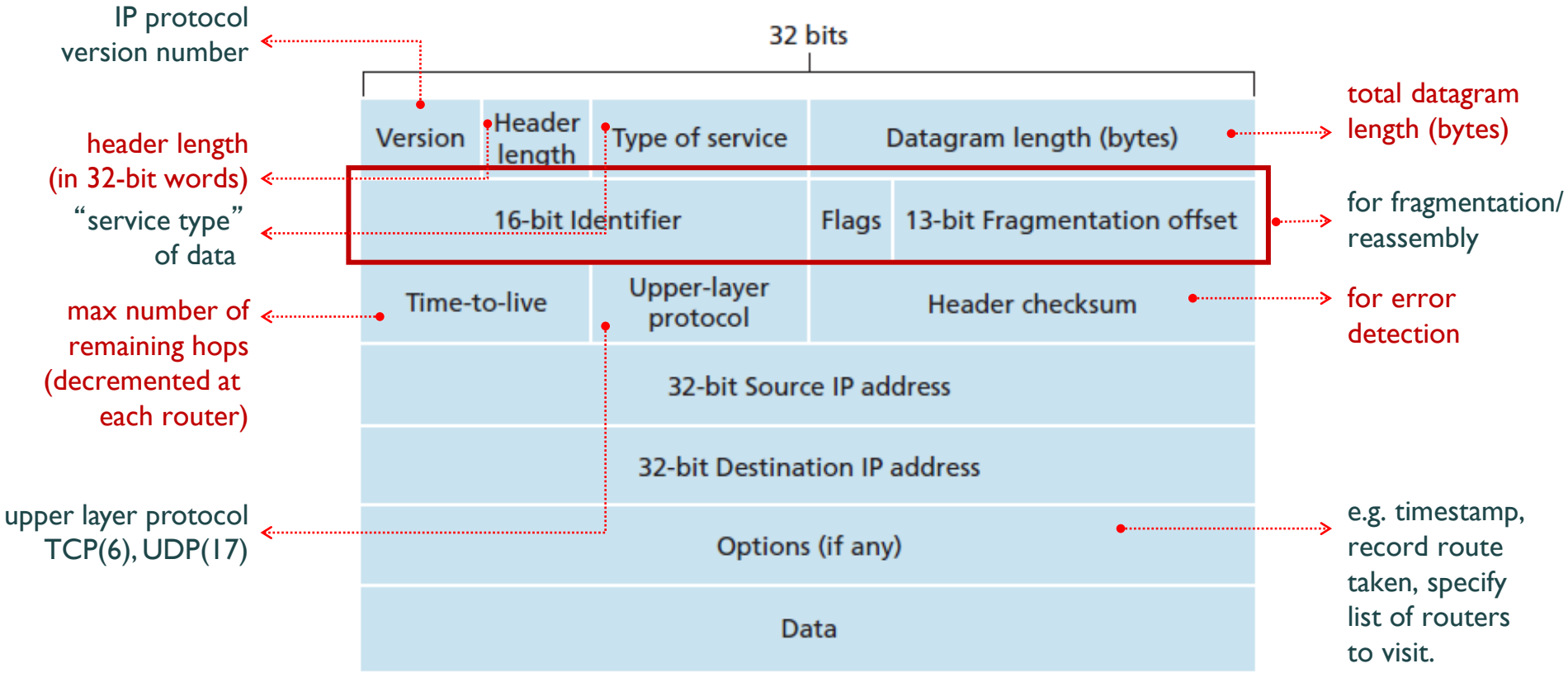
❖ TCP Segment header



Network Layer

❖ IPv4 Datagram (Packet) Format

- the datagram plays a central role in the Internet

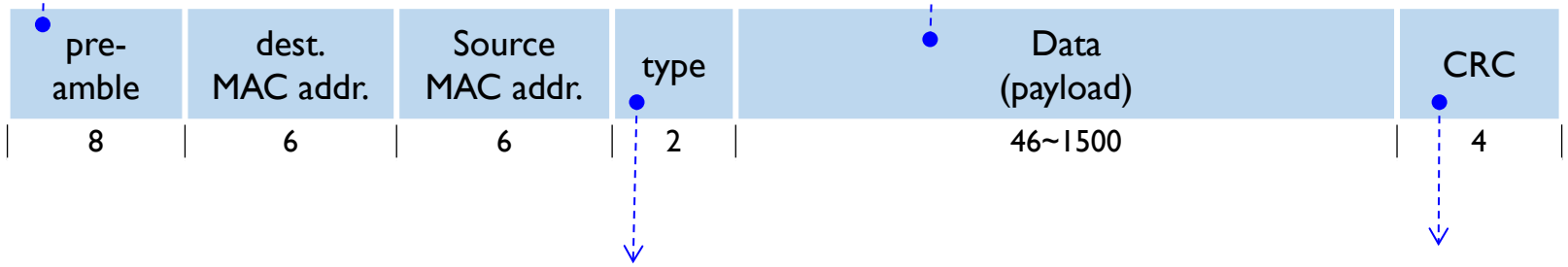


Datalink Layer

❖ Ethernet II frame structure (DIX)

• Preamble
- used to synchronize receiver and sender clock rates
- 7 bytes with pattern 10101010 followed by one byte with 10101011

• Data
- upper layer SDU
- MTU of Ethernet =1500 bytes
- if data size is less than 46 bytes, it should be stuffed

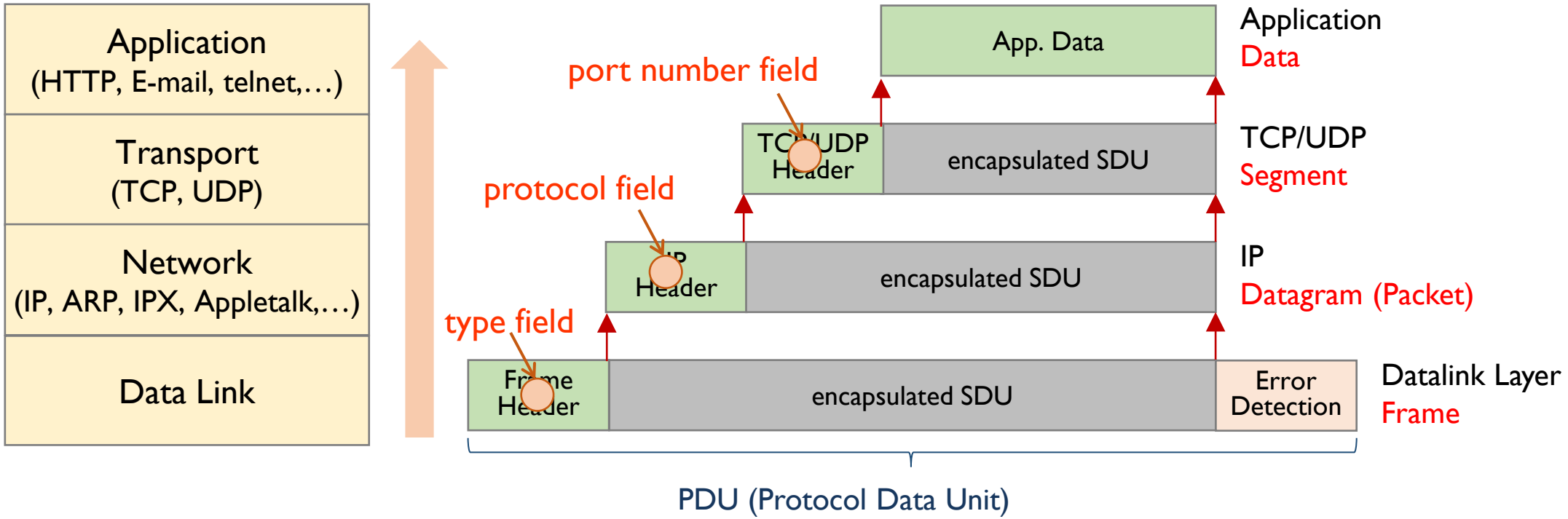


• type:
- to indicate higher layer protocol (0x0800 for IP, …)

• CRC:(cyclic redundancy check)
- for error detection
- if error detected, frame is dropped

TCP/IP – Decapsulation

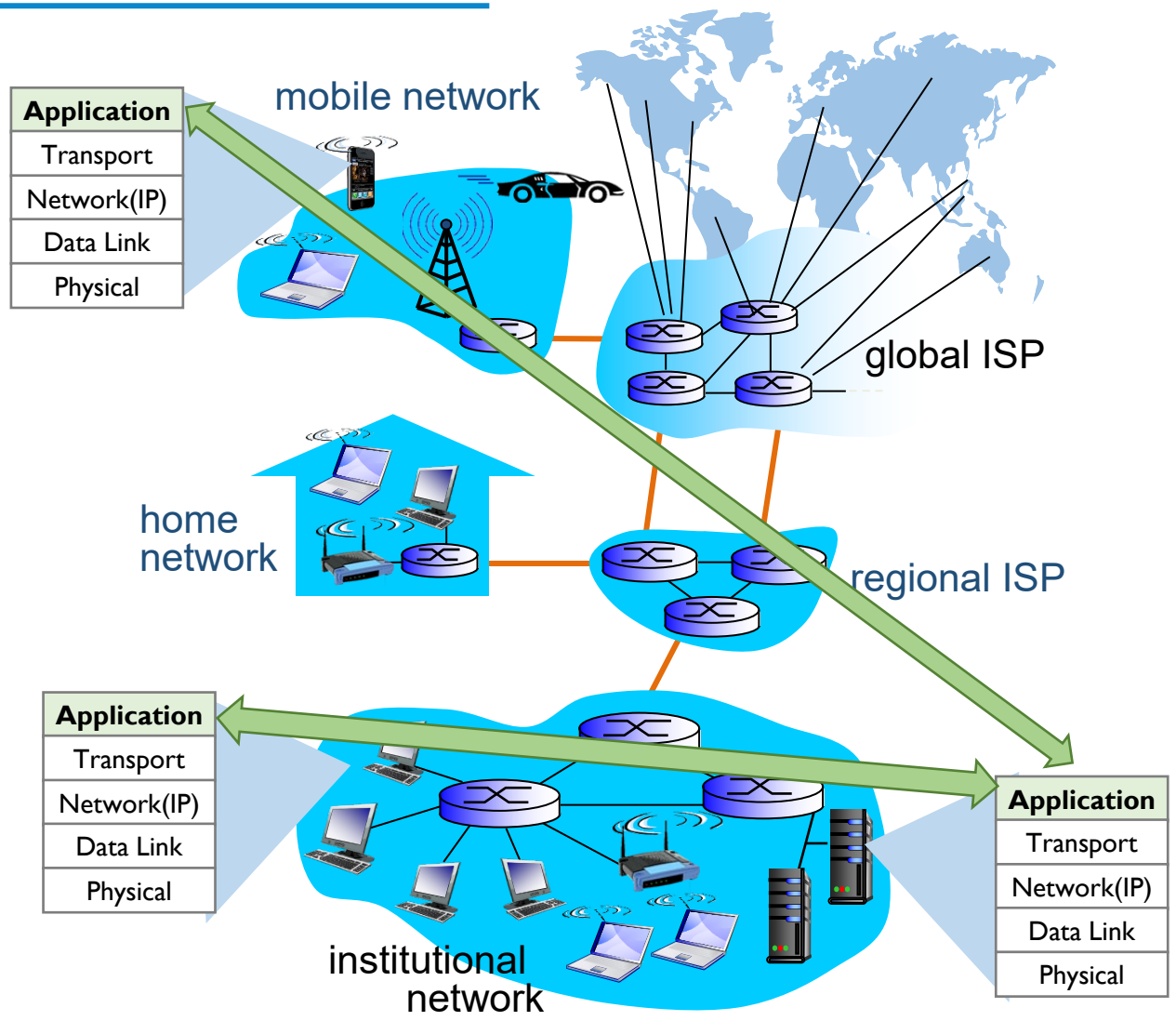
- Interfacing towards upper layers
  - each layer
    - has a specific field in its header
    - to indicate the upper layer protocol for the SDU to be delivered



SOCKET BASICS

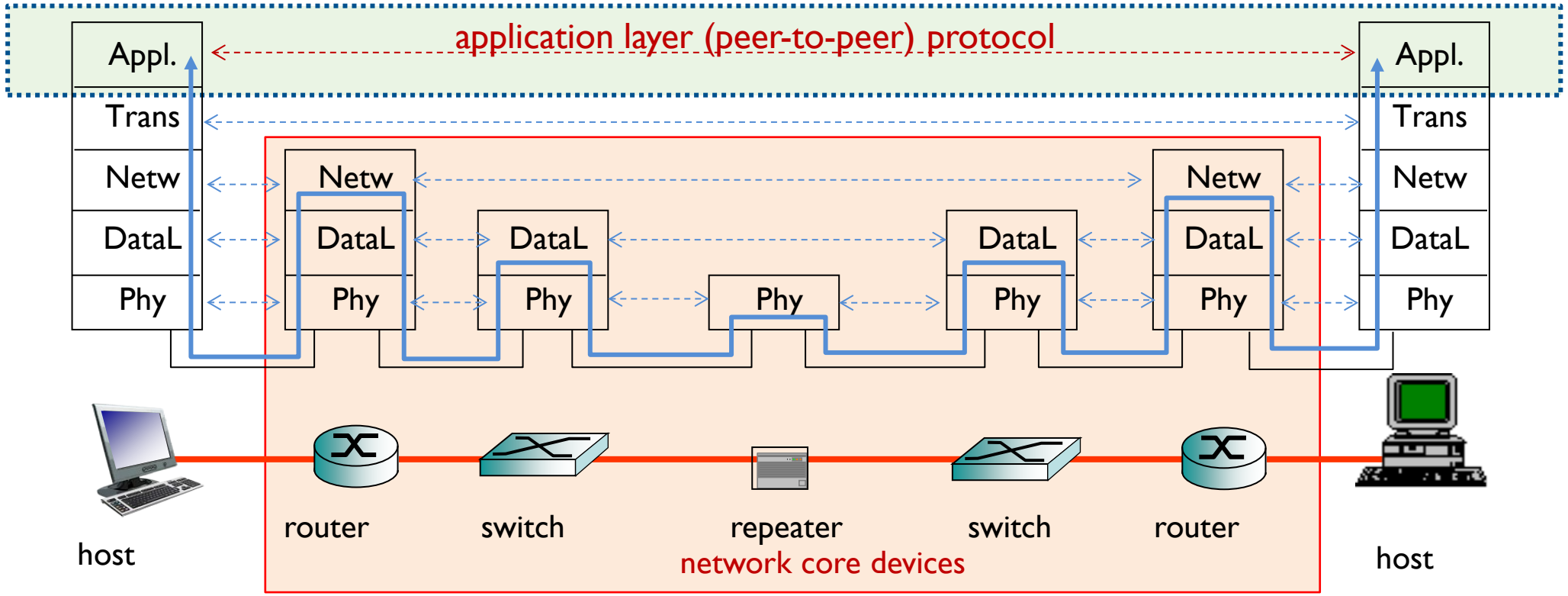
Network Application Programming

- Application Layer
  - the interface to the network
  - the means for converting communication to data that can be transferred across the data network
- Application programs
  - run on (different) end systems,
  - communicate over network,
  - but no need to write software for network-core devices



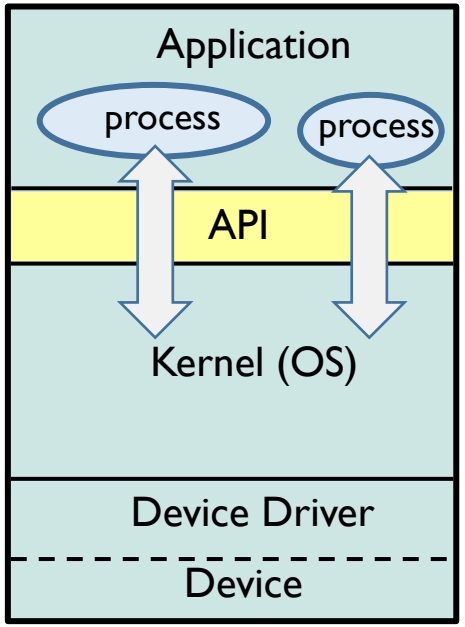
Network Application Programming

- Why "no need to write application programs for network-core devices" ?
  - Protocol means (peer-to-peer) protocol

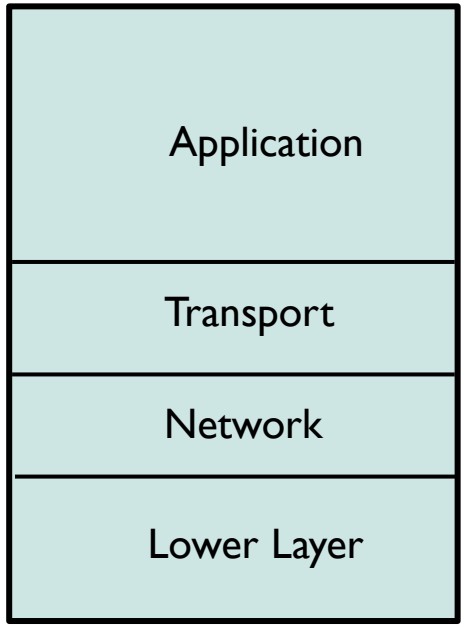


Network Application Programming

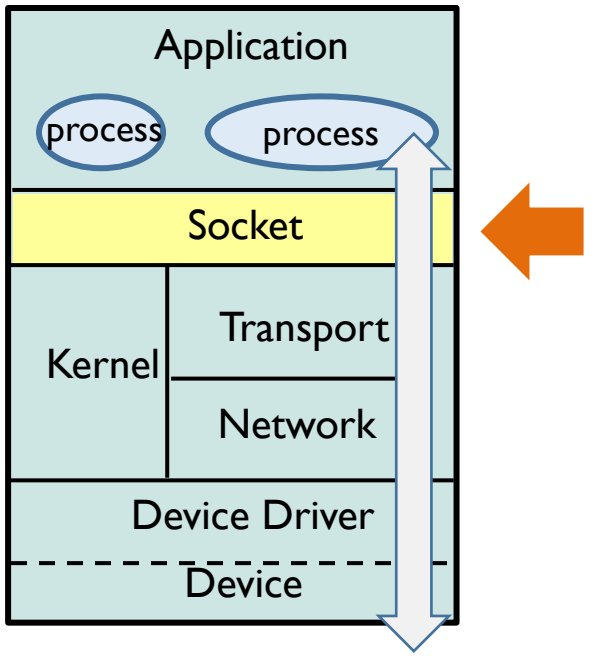
- Process, API, and socket
  - process: application program running within a host
  - API : application programming interface between process and OS
  - socket : network programming interface between process and the network protocols



OS



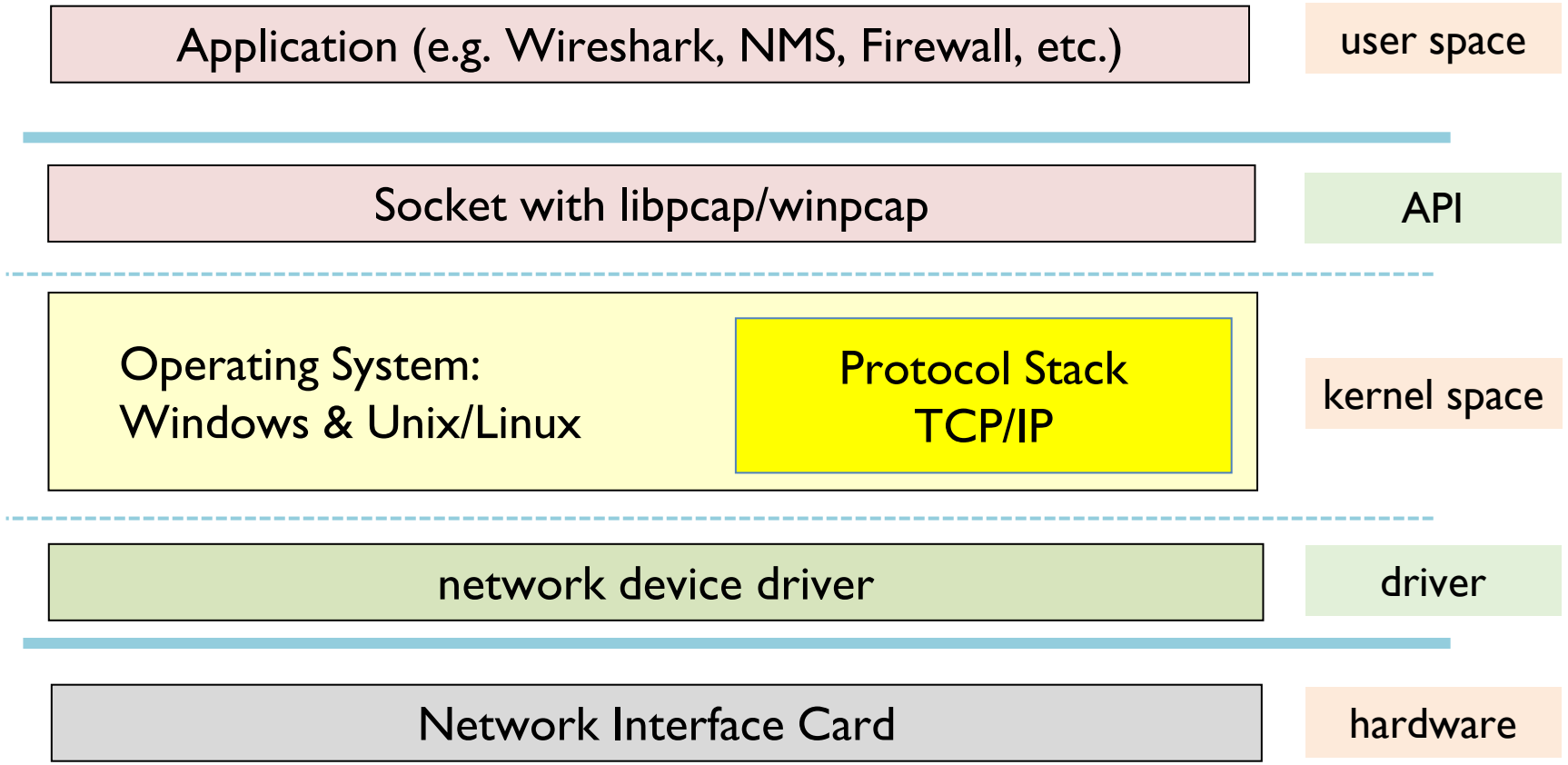
TCP/IP



Network Programming

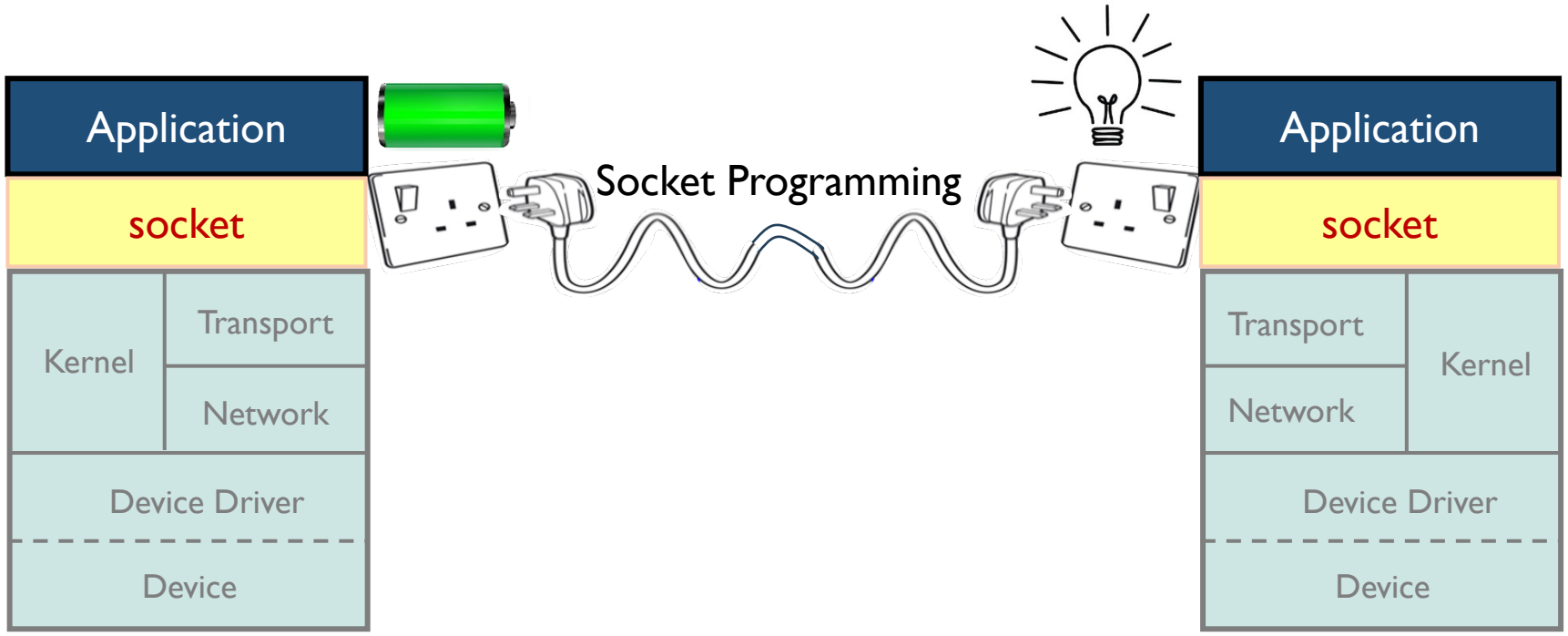
Network Application Programming

❖ Application, OS, Device and Socket



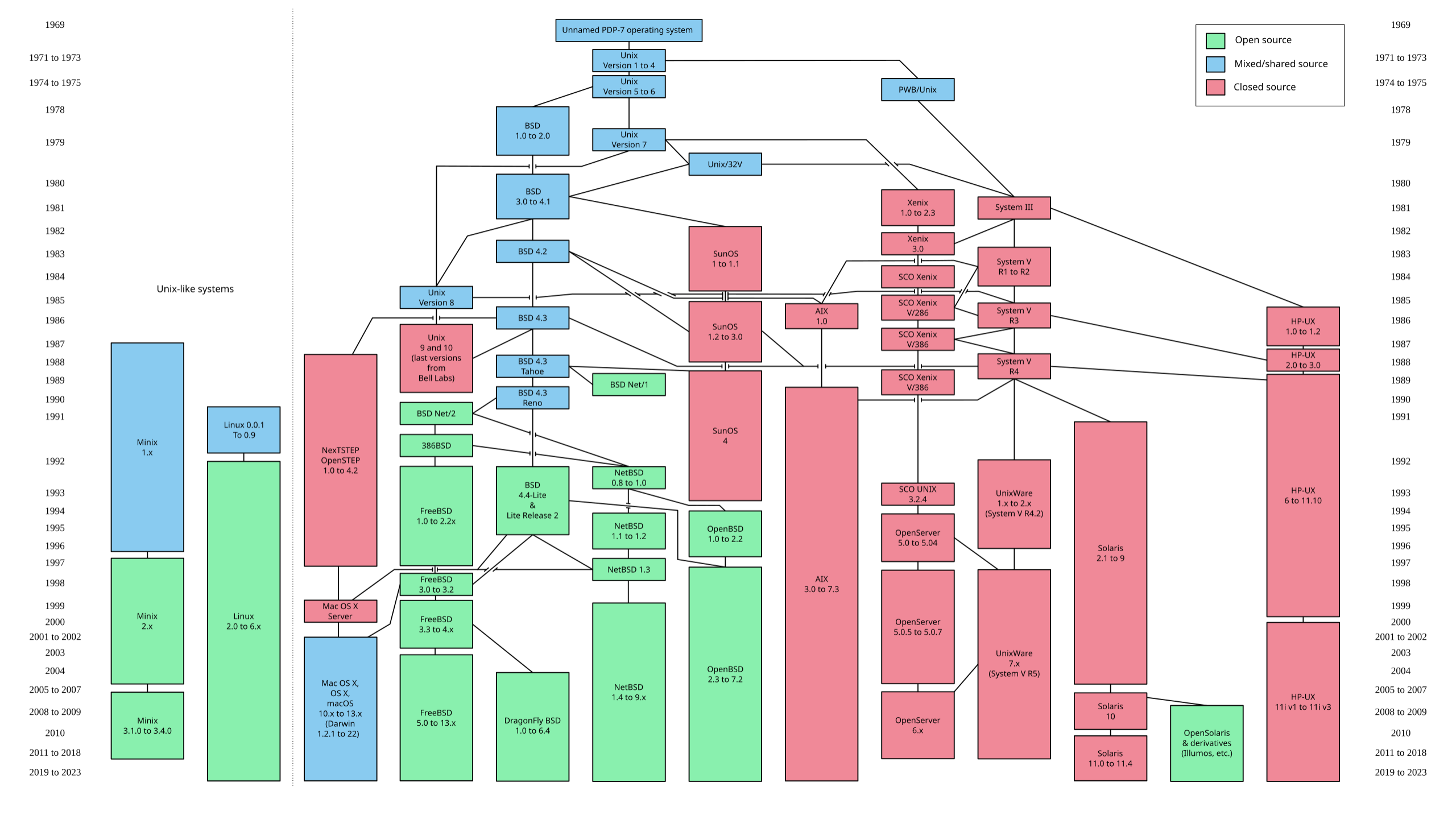
Network Application Programming

- Sockets provide
  - network programming interface between
    - processes at application layers on the network
    - through TCP/IP network kernel (OS), but don't care of it



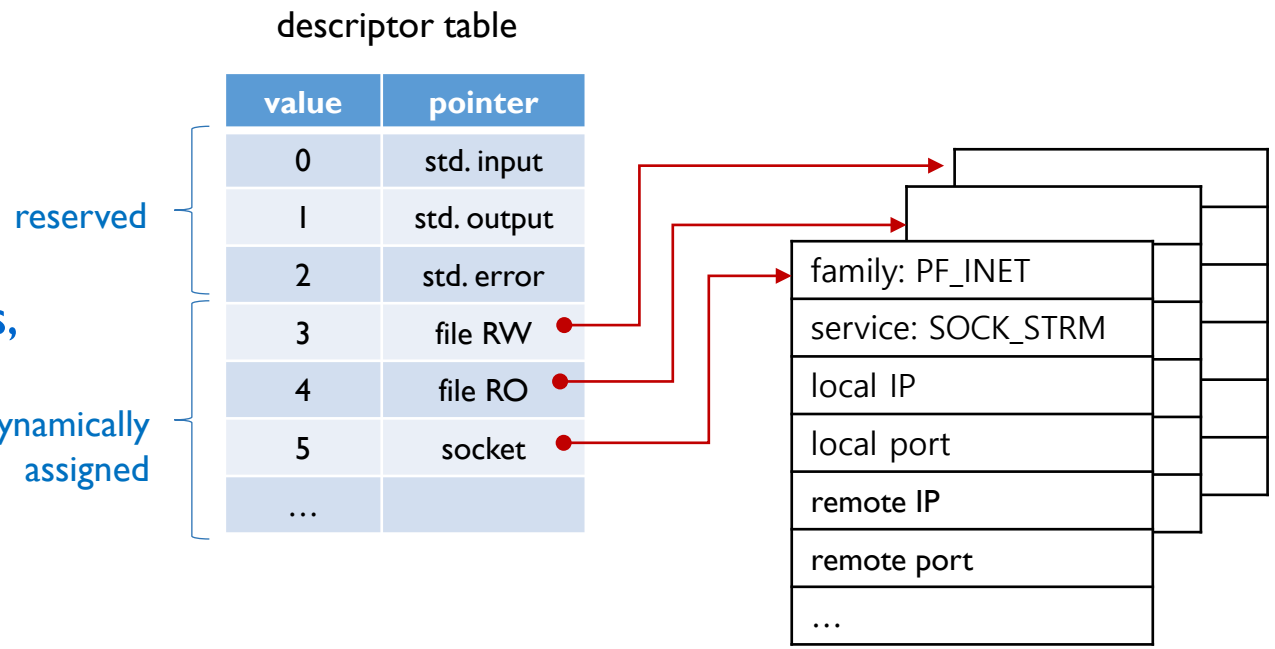
Socket History

- History Of Socket
  - 1969: UNIX
  - 1975: UNIX implementation of IP applications by University of Illinois
    - for FTP and Telnets
    - complicated and performance problem
  - 1982: UNIX 4.1BSD (Berkeley Software Distribution) version
    - included TCP/IP and Socket
    - by integrating sockets with UNIX OS's file descriptors
  - 1986: UNIX 4.3 BSD version
    - widely used TCP/IP and Socket
  - 1988: First release of tcpdump + libpcap
    - by Van Jacobson, Sally Floyd, Vern Paxson and Steven McCanne
  - 1991: First Linux (ver 0.02) has been introduced

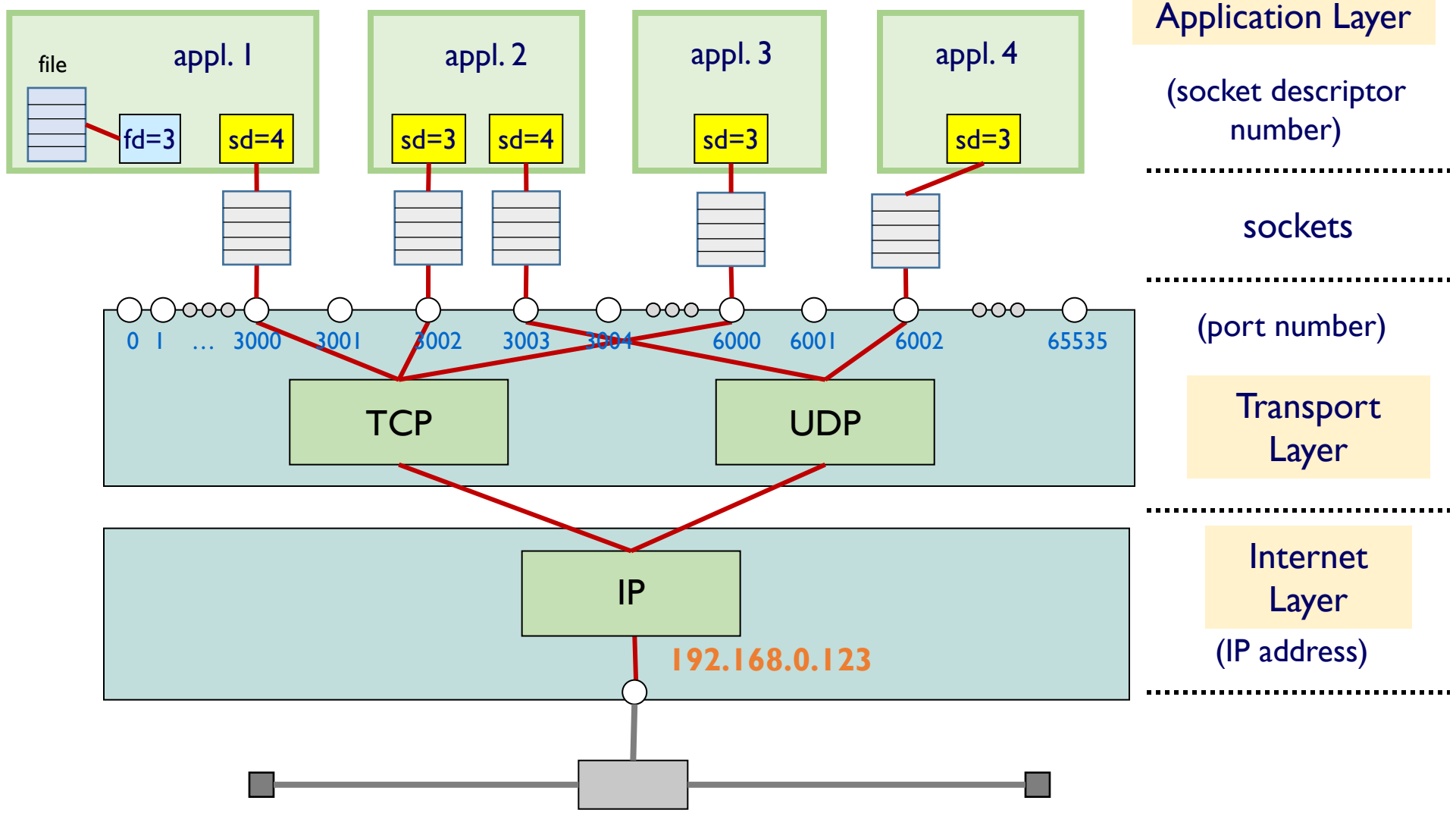

Unix-like systems
1969
1971 to 1973
1974 to 1975
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001 to 2002
2003
2004
2005 to 2007
2008 to 2009
2010
2011 to 2018
2019 to 2023
Open source
Mixed/shared source
Closed source
Unnamed PDP-7 operating system
Unix Version 1 to 4
Unix Version 5 to 6
Unix Version 7
PWB/Unix
BSD 1.0 to 2.0
Unix/32V
BSD 3.0 to 4.1
Xenix 1.0 to 2.3
System III
Xenix 3.0
BSD 4.2
SunOS 1 to 1.1
System V R1 to R2
SCO Xenix
Unix Version 8
SCO Xenix V/286
AIX 1.0
BSD 4.3
SunOS 1.2 to 3.0
System V R3
HP-UX 1.0 to 1.2
SCO Xenix V/386
HP-UX 2.0 to 3.0
Unix 9 and 10 (last versions from Bell Labs)
BSD 4.3 Tahoe
System V R4
BSD Net/1
SCO Xenix V/386
BSD 4.3 Reno
NexTSTEP OpenSTEP 1.0 to 4.2
BSD Net/2
SunOS 4
Minix 1.x
Linux 0.0.1 To 0.9
386BSD
NetBSD 0.8 to 1.0
HP-UX 6 to 11.10
FreeBSD 1.0 to 2.2x
BSD 4.4-Lite & Lite Release 2
SCO UNIX 3.2.4
UnixWare 1.x to 2.x (System V R4.2)
NetBSD 1.1 to 1.2
OpenBSD 1.0 to 2.2
OpenServer 5.0 to 5.04
Solaris 2.1 to 9
NetBSD 1.3
FreeBSD 3.0 to 3.2
AIX 3.0 to 7.3
Minix 2.x
Linux 2.0 to 6.x
Mac OS X Server
FreeBSD 3.3 to 4.x
OpenServer 5.0.5 to 5.0.7
UnixWare 7.x (System V R5)
Mac OS X, OS X, macOS 10.x to 13.x (Darwin 1.2.1 to 22)
NetBSD 1.4 to 9.x
OpenBSD 2.3 to 7.2
Solaris 10
HP-UX 11i v1 to 11i v3
Minix 3.1.0 to 3.4.0
FreeBSD 5.0 to 13.x
DragonFly BSD 1.0 to 6.4
OpenServer 6.x
OpenSolaris & derivatives (Illumos, etc.)
Solaris 11.0 to 11.4

Socket Descriptor

- descriptor
  - an abstract indicator
    - used to access a file or other I/O resources (e.g. pipe, network socket, …)
    - only valid in an application program (or process)
  - fd 0, 1, and 2 are reserved
    - 0 for standard input (e.g. keyboard)
    - 1 for standard output
    - 2 for standard error
  - other descriptors (≥ 3)
    - are dynamically assigned for user files,
    - sockets …

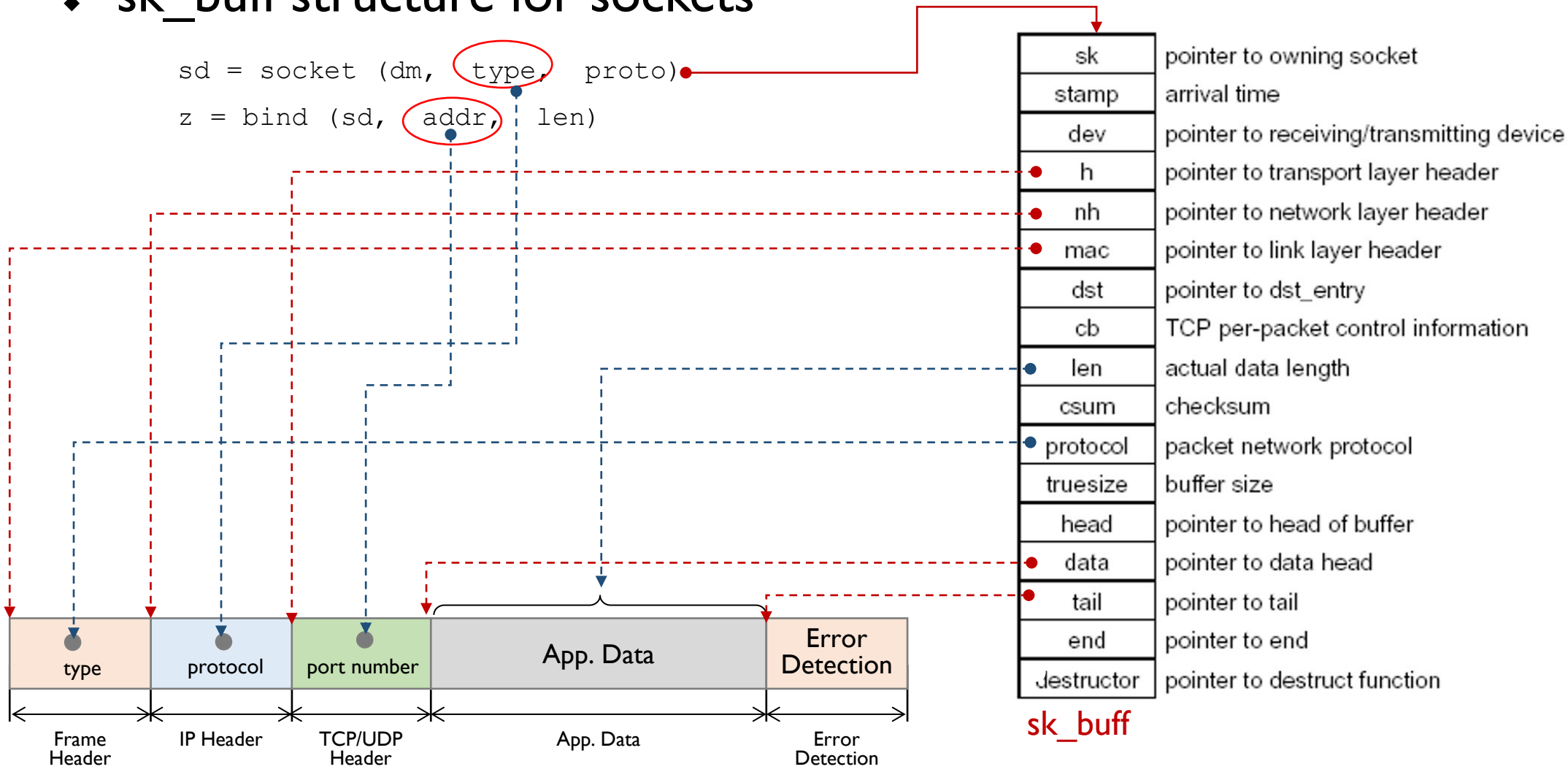


Applications, Sockets and TCP/IP

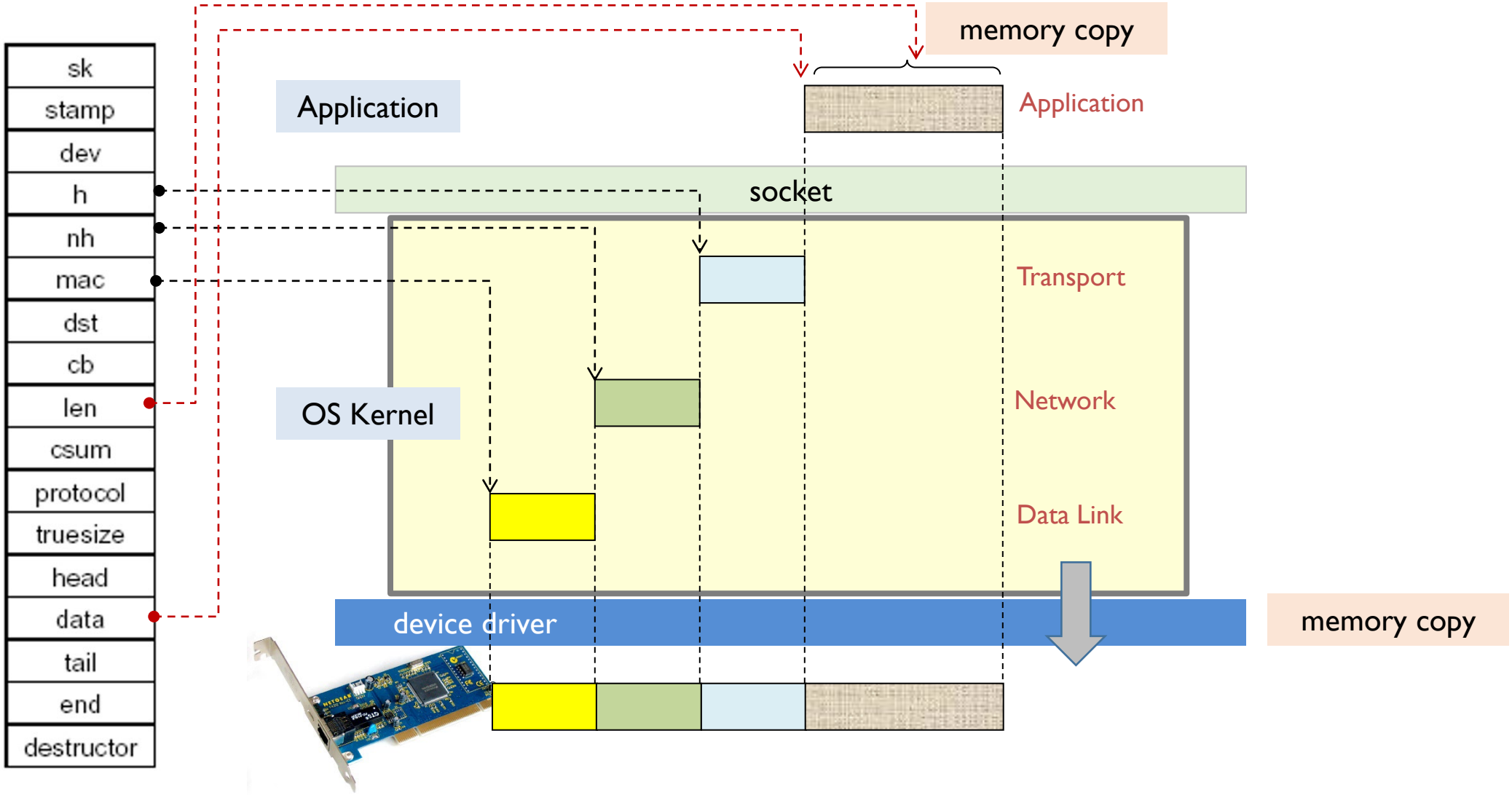


Socket and Encapsulation Process

- sk_buff structure for sockets

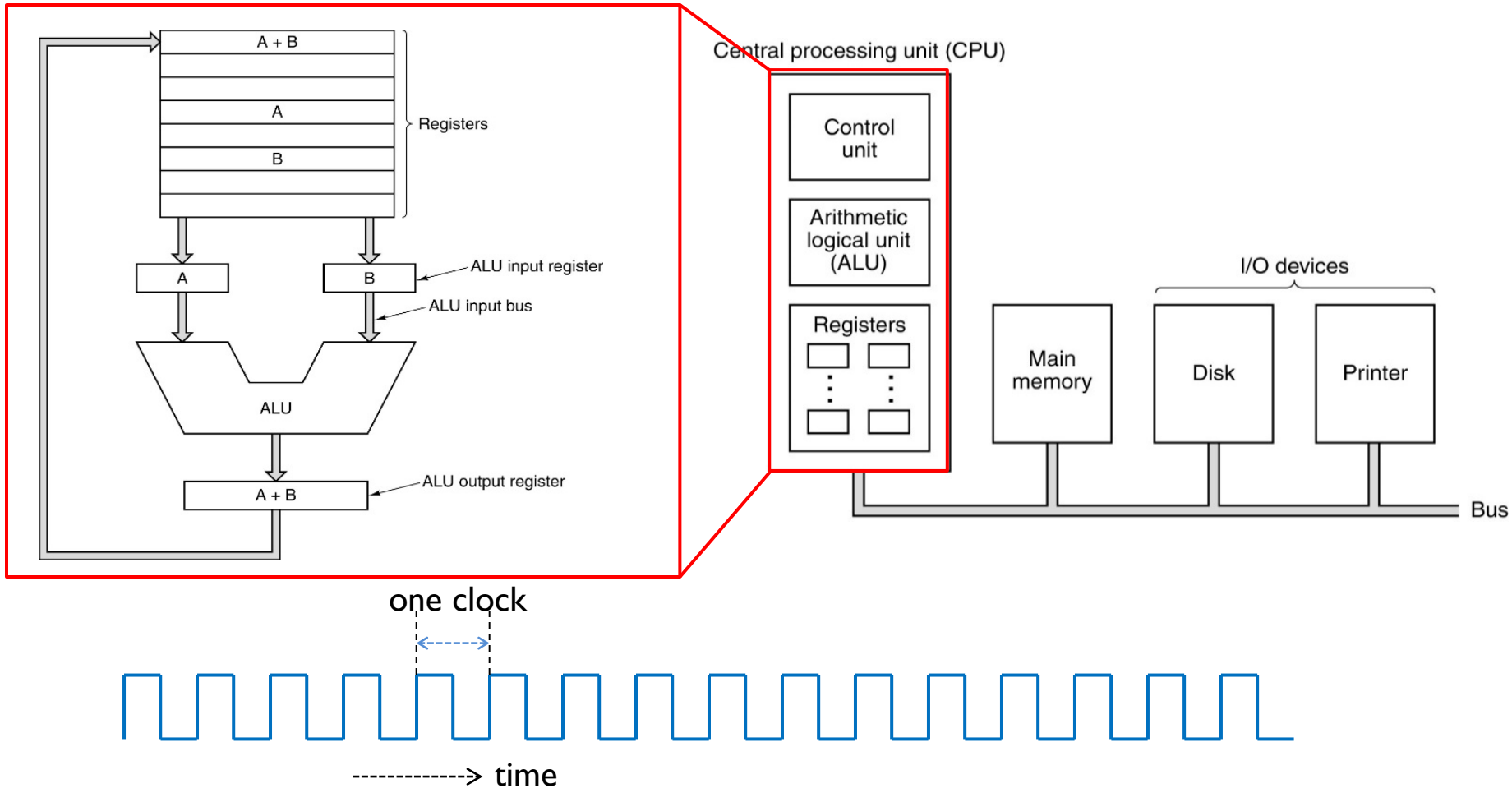


Socket and Encapsulation Process



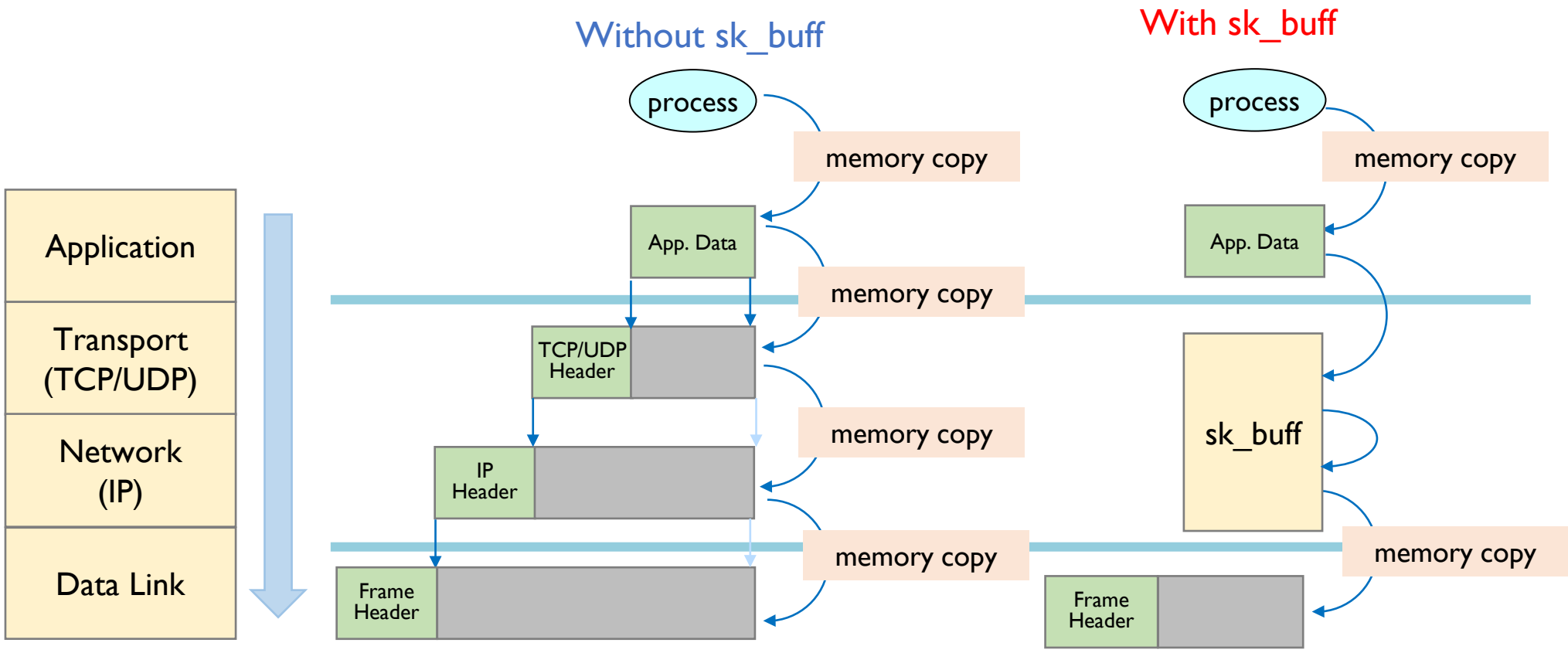
Socket and Encapsulation Process

- Encapsulation Memory Copy Issue



Socket and Encapsulation Process

❖ Encapsulation Memory Copy Issue

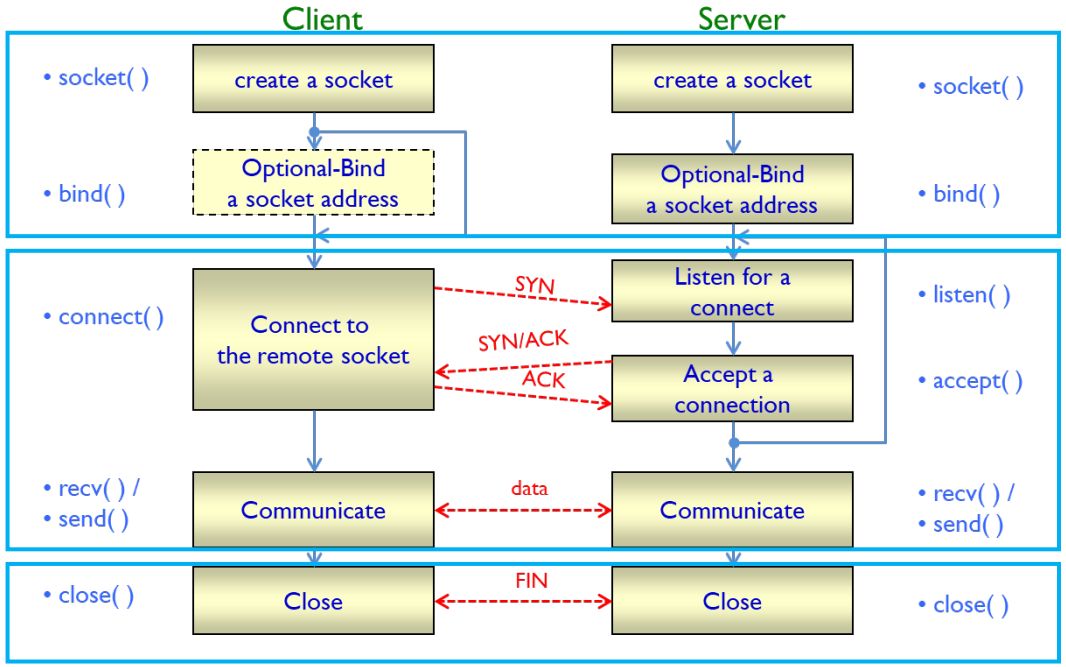


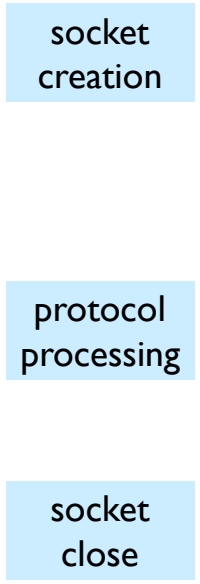
SOCKET FUNCTIONS

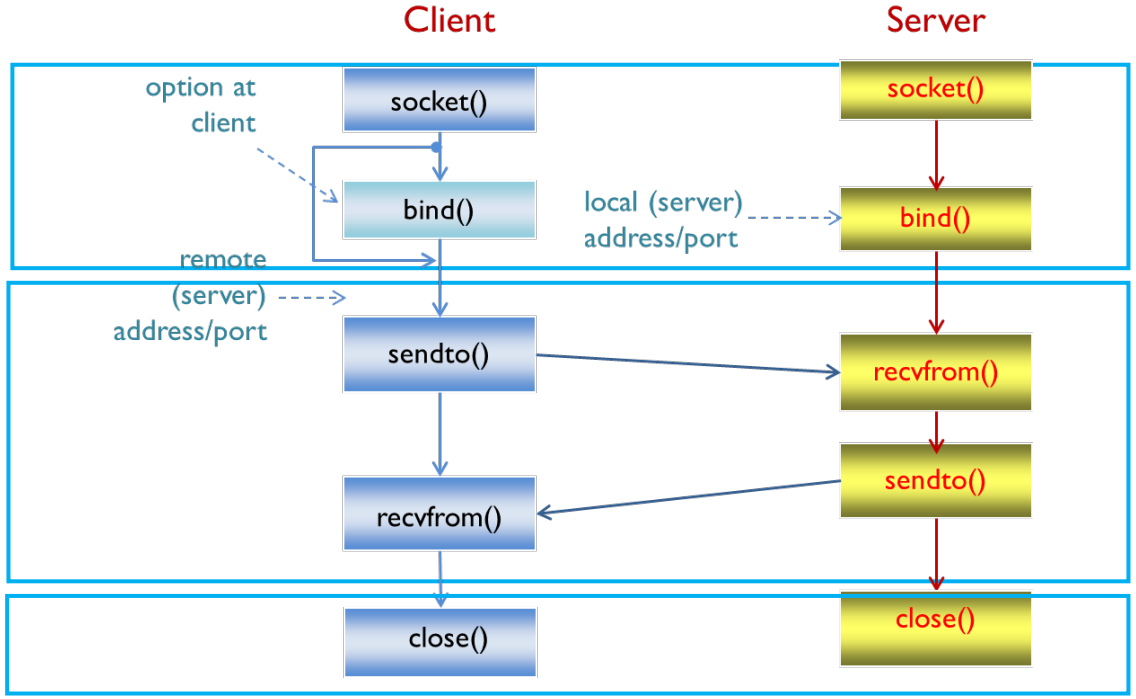
Socket Operation

Socket의 Flow임.

- Basic Operations Using Sockets
  - Creating Sockets
  - Performing I/O on Sockets
  - Closing Sockets





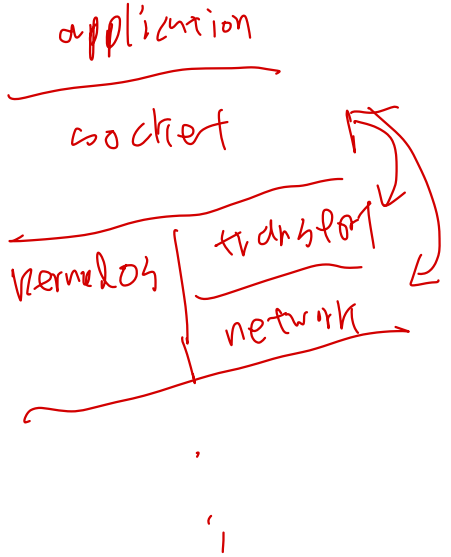


basic TCP-based network programming

basic UDP-based network programming

Socket Creation (1/4)

- Five elements for creating a socket
  - Transport Layer Protocol (TCP or UDP)
  - Local IP Address
  - Local Port Number
  - Remote IP Address
  - Remote Port Number



Socket Creation (2/4)

❖ socket ( ) function

```
#include <sys/socket.h>
int socket ( int domain, int type, int protocol) ;
```

- domain: specify address family
  - AF_INET (for IPv4 system)
  - AF_INET6 (for IPv6 system)
  - AF_UNIX (for UNIX)
  - etc
- type: specify socket type for communication semantics
  - SOCK_STREAM (for stream-oriented, e.g. TCP)
  - SOCK_DGRAM (for datagram-based, e.g. UDP)
  - SOCK_RAW (for raw-mode without transport or network layers, e.g. ICMP…)
- protocol: specify a particular protocol to be used with the socket
  - 0 by default
- return
  - file descriptor on success
  - -1 on error (with errno)

Socket Creation (3/4)

❖ Socket types and protocol

/usr/include/bits/socket.h

```
/* Types of sockets.  */
enum __socket_type
{
SOCK_STREAM = 1,        /* Connection-based, e.g. TCP */
SOCK_DGRAM = 2,         /* Connectionless, e.g. UDP */
SOCK_RAW    = 3,        /* Raw protocol interface.  e.g. ICMP */
SOCK_RDM    = 4,        /* Reliably-delivered messages.  */
SOCK_SEQPACKET = 5,     /* Connection-based, on
                           non-Internet protocols (e.g. X.25) */
SOCK_PACKET = 10        /* Linux specific way of getting packets
                           at the dev level. */
};
```

/usr/include/netinet/in.h

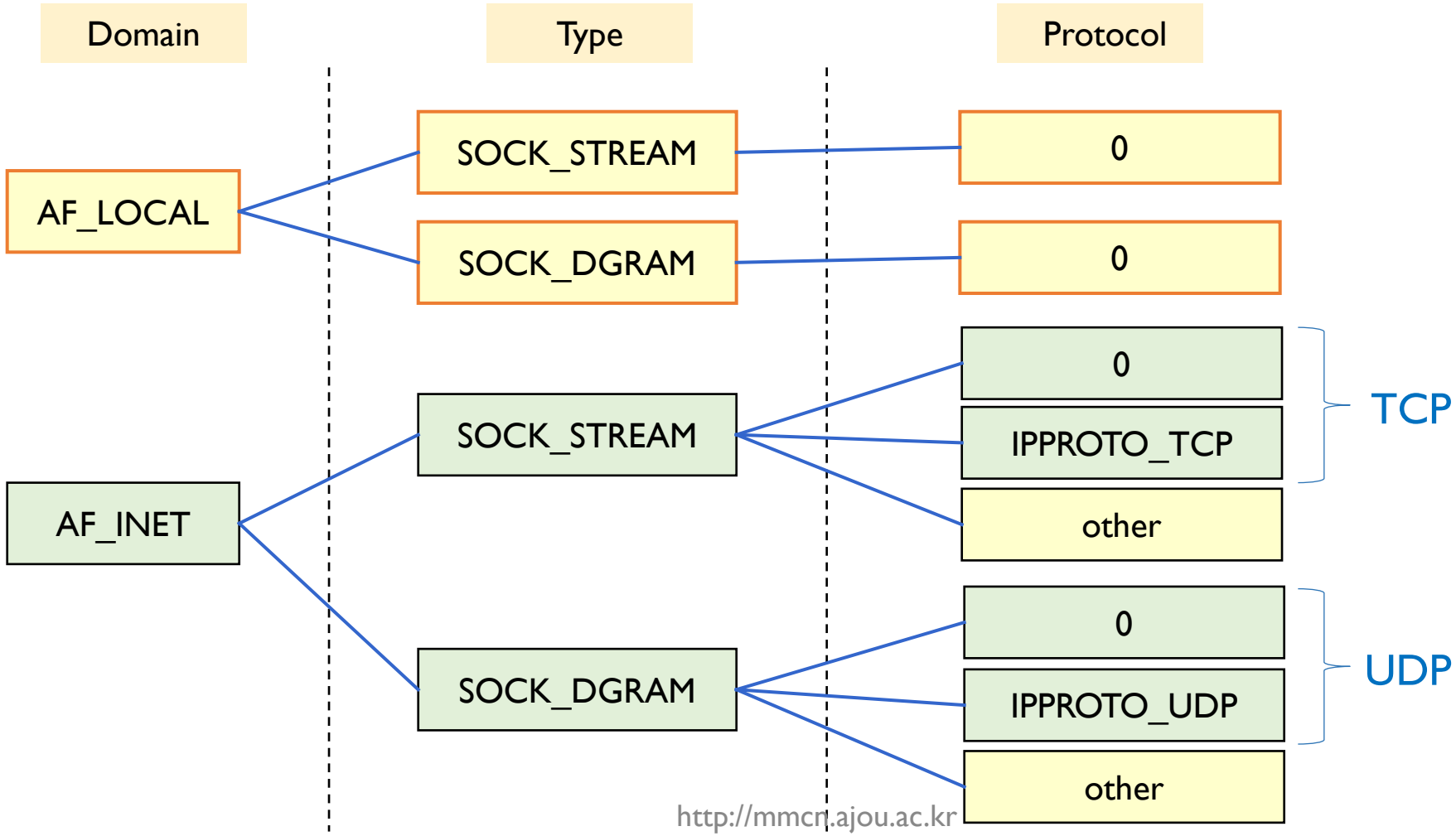
```
/* Standard well-defined IP protocols.  */
enum
{
IPPROTO_IP      = 0,    /* Dummy protocol.  */
IPPROTO_ICMP = 1,       /* Internet Control Message Protocol.  */
IPPROTO_IGMP = 2,       /* Internet Group Management Protocol. */
IPPROTO_TCP   = 6,      /* Transmission Control Protocol.  */
IPPROTO_UDP  = 17,      /* User Datagram Protocol.  */
….
}
```

- when type is set to 0,
  - OS Kernel will choose the correct protocol value.

```
int s = socket ( AF_INET,  SOCK_DGRAM,  0 );
```

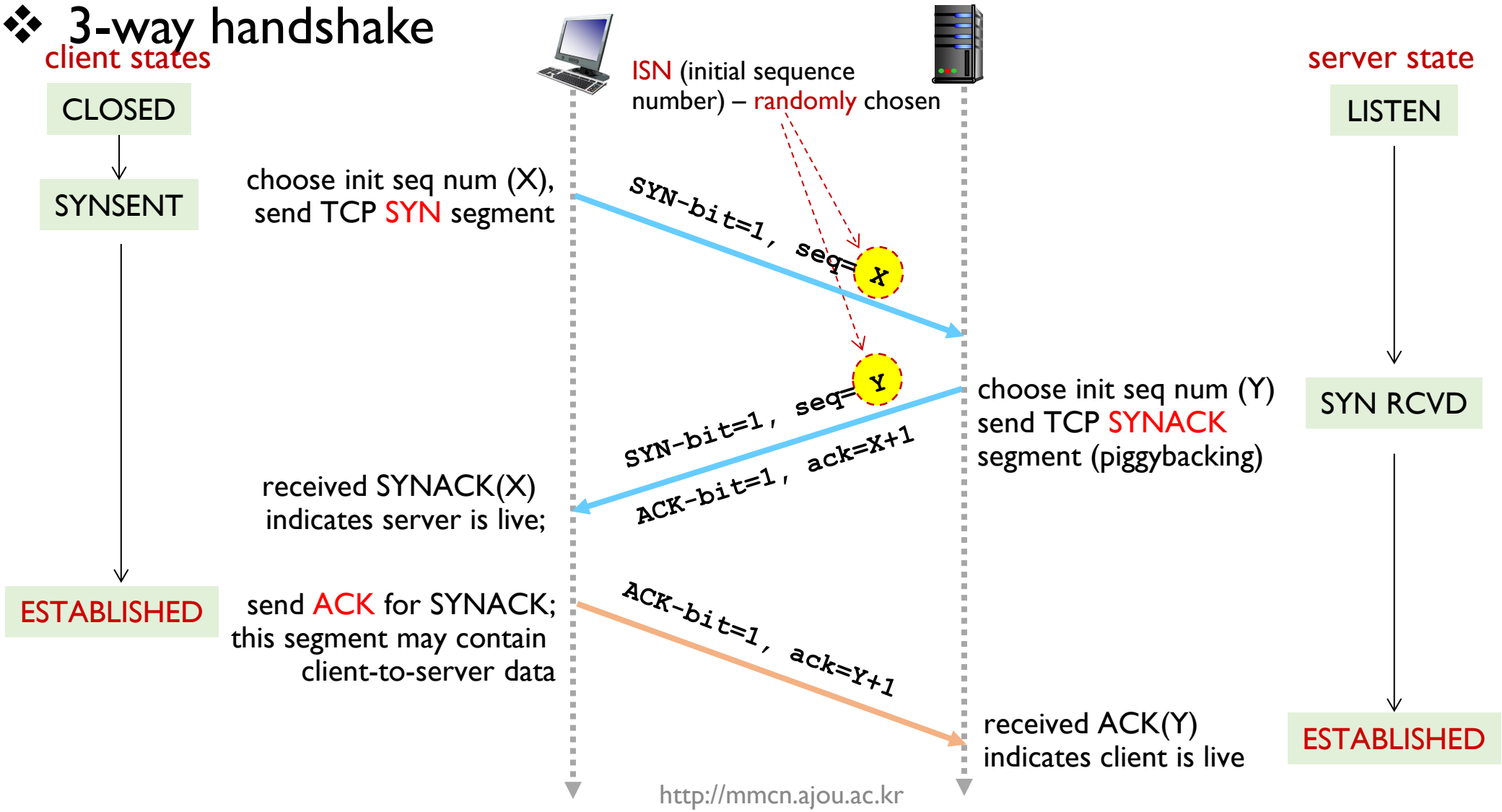
Socket Creation (4/4)

❖ Socket Domain, Type, and Protocol



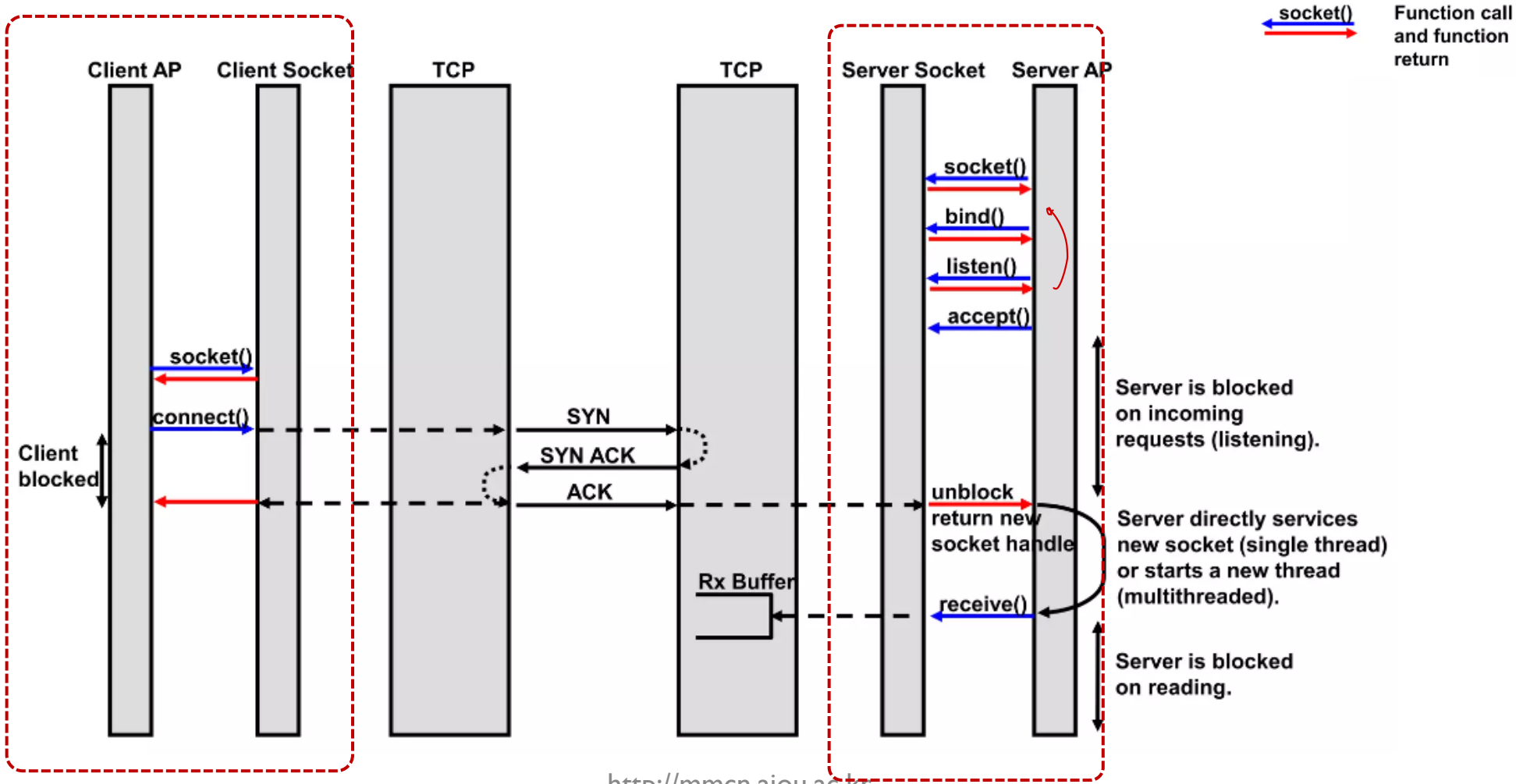
TCP Connection Setup (1/2)

❖ 3-way handshake



TCP Connection Setup (2/2)

- Socket programming viewpoints

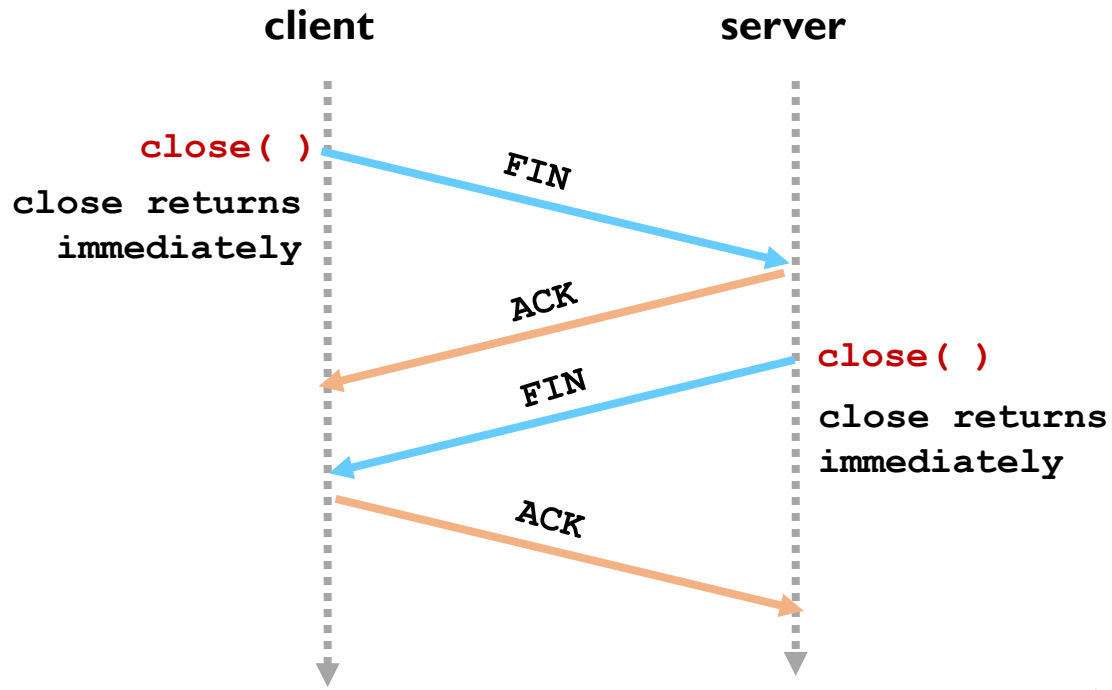


Closing Sockets (1/4)

- close ( ) function
  - to close immediately a socket

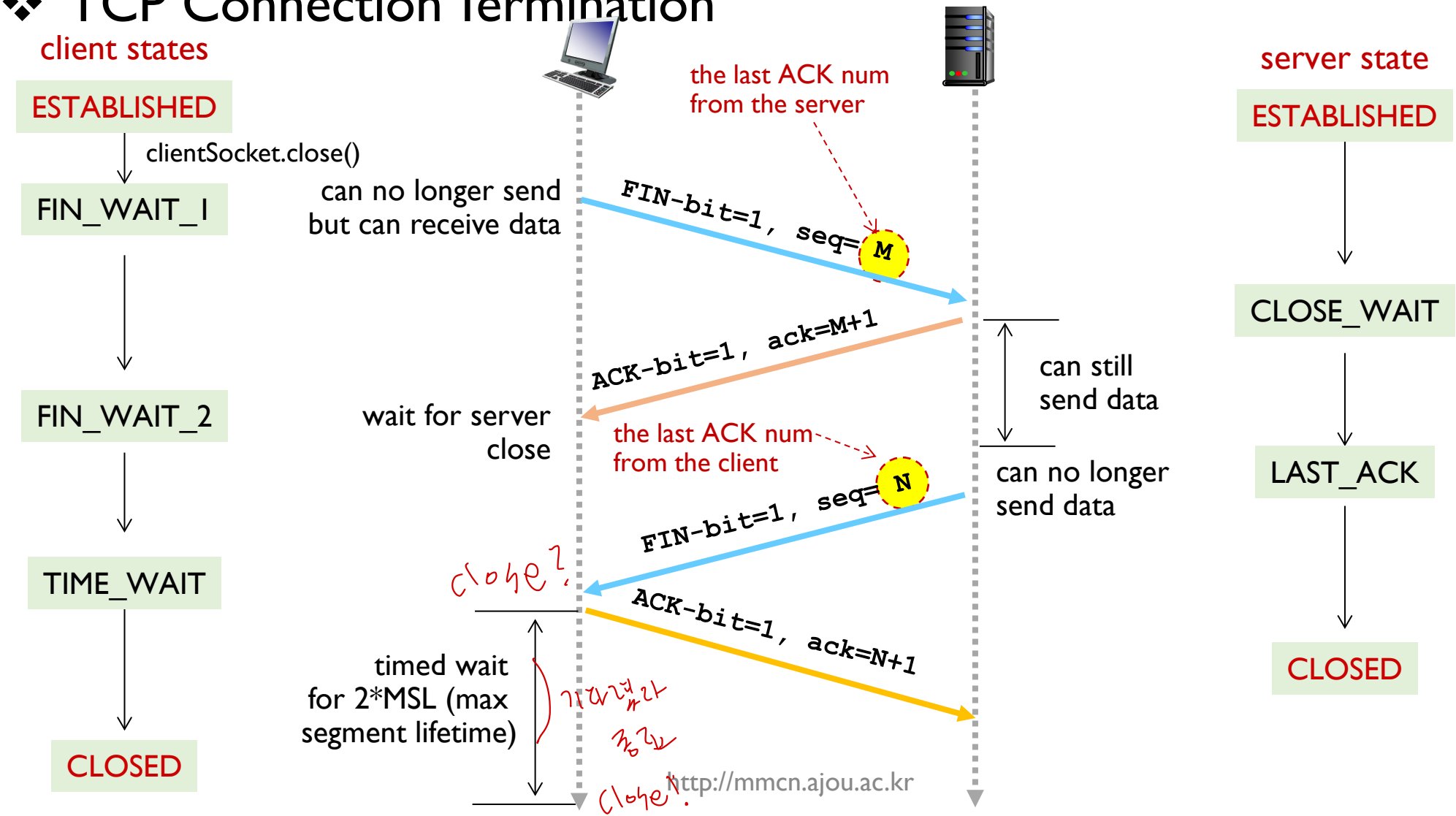
```
#include <stdio.h>
int close (int s);
```

- s: socket number to be closed
- return
  - 0 for successful close completion
  - -1 on error (with errno)



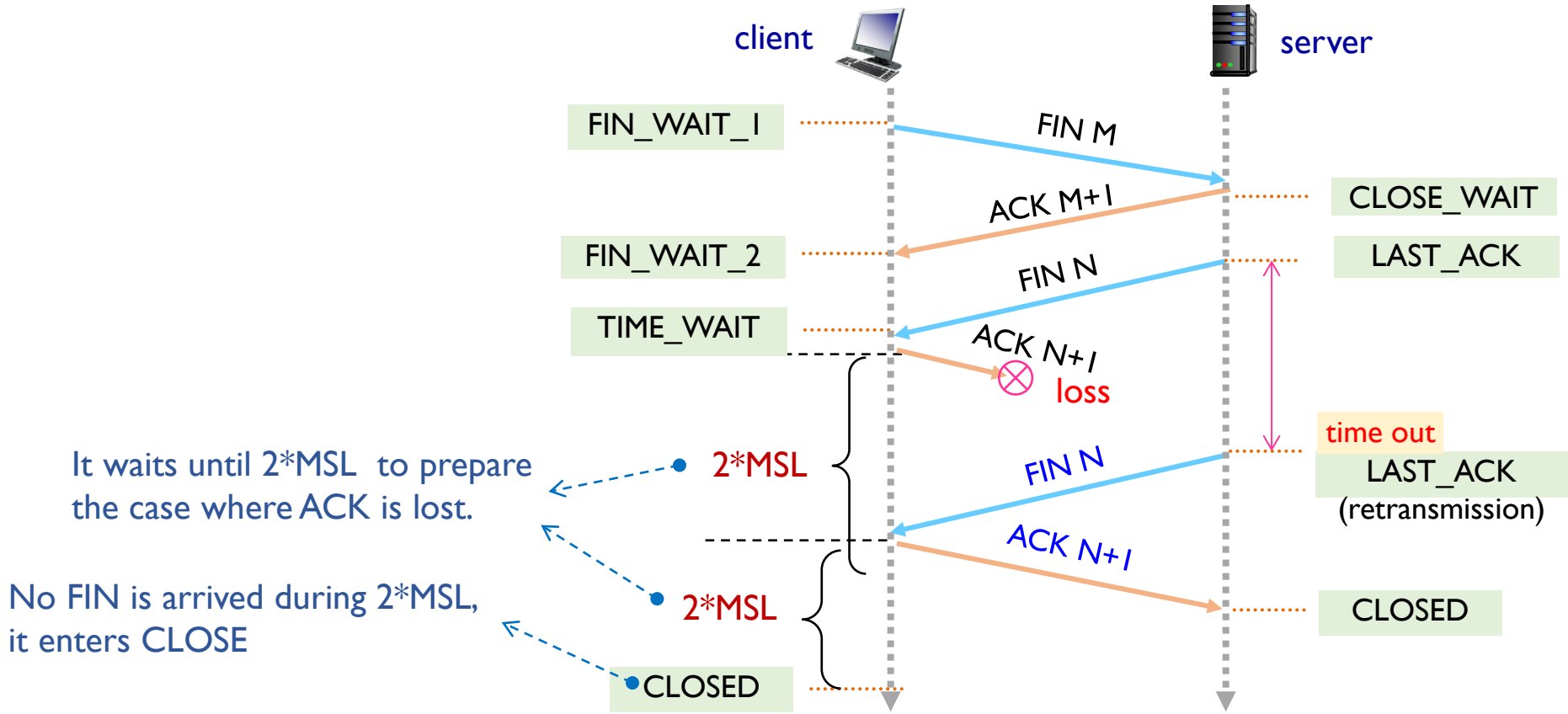
Closing Sockets (2/4)

❖ TCP Connection Termination



Closing Sockets (3/4)

- Maximum Segment Lifetime (MSL)
  - the time a TCP segment can exist in the internetwork system
    - MSL = 2 min. default in RFC 793, 30 sec or 1 min. for common implementation



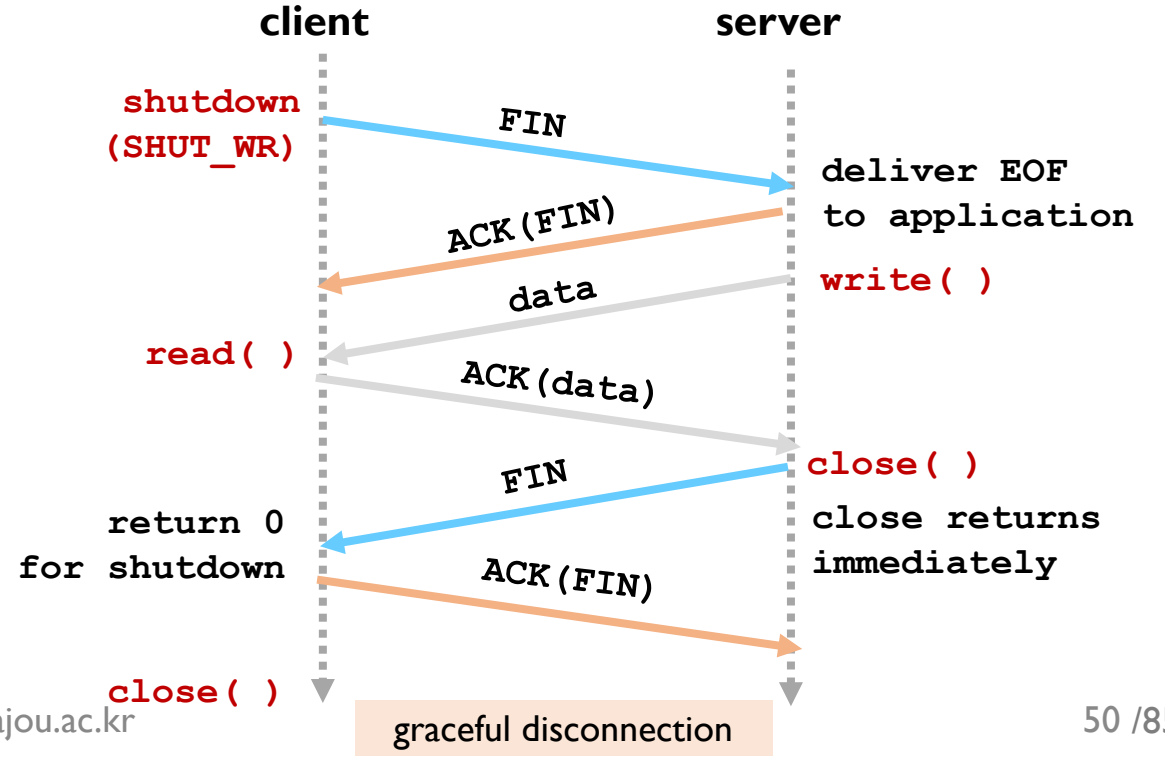
Closing Sockets (4/4)

❖ shutdown ( ) function

- to close a part of transmission or reception on a full-duplex socket

```
#include <sys/socket.h>
int shutdown (int s, int how);
```

- s: socket number to be closed
- how
  - SHUT_RD 보내는거 close
  - SHUT_WR 받는거 close
  - SHUT_RDRW
- return
  - 0 for successful close completion
  - -1 on error (with errno)



Socket Functions (1/3)

- Socket-Specific
  - socketpair
  - socket
  - bind
  - connect
  - listen
  - accept
- Socket Addressing
  - getsockname
  - getpeername
- Reading Of Sockets
  - read
  - readv
  - recv
  - recvfrom
  - recvmsg
- Writing To Sockets
  - write
  - writev
  - send
  - sendto
  - sendmsg

Socket Functions (2/3)

- Other Socket I/O
  - select
  - cmsg
- Controlling Sockets
  - shutdown
  - getsockopt
  - setsockopt
  - dup
  - dup2
  - fcntl
  - ioctl
- Network Supported
  - byteorder
  - inet_addr
  - inet_aton
  - inet_ntoa
  - inet_network
  - inet_lnaof
  - inet_netof
  - inet_makeaddr
  - getservent
  - getprotoent

Socket Functions (3/3)

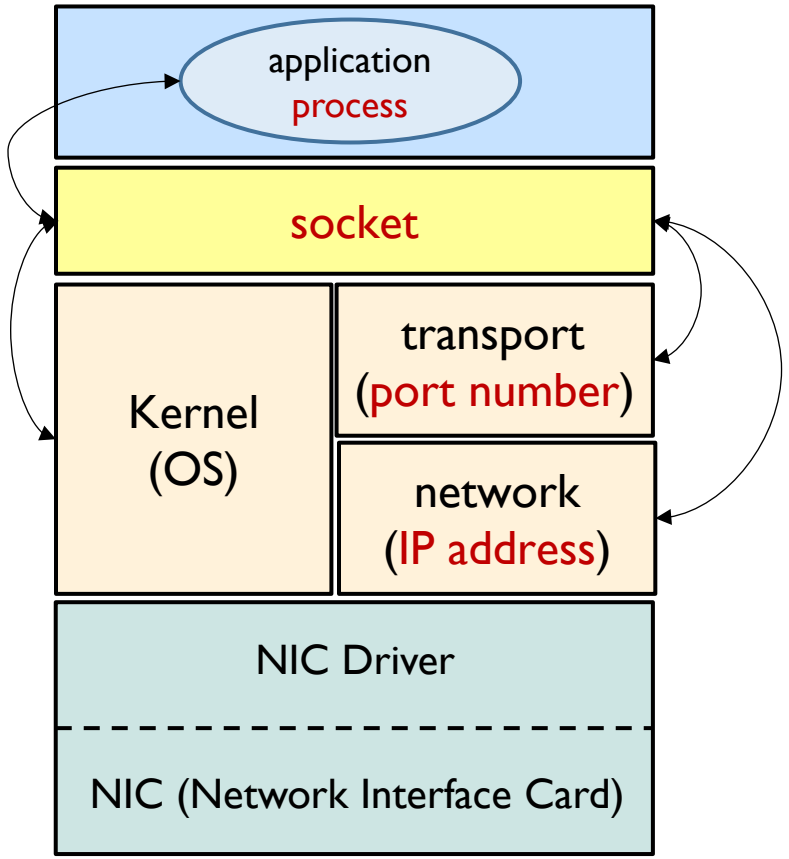
- Standard I/O Support
  - fdopen
  - fileno
  - setbuf
  - setbuffer
  - setlinebuf
  - setvbuf

- Hostname Support
  - uname
  - gethostname

SOCKET ADDRESS

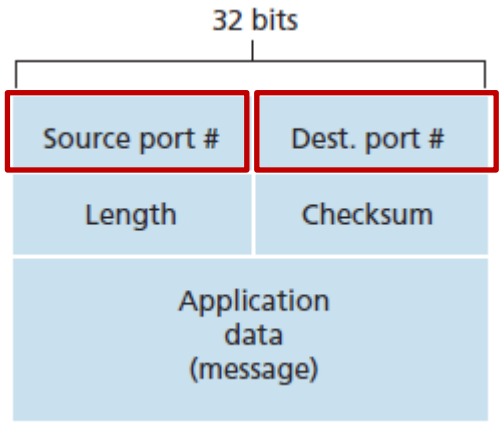
Socket Address

- Socket is
  - a software endpoint of a two-way communication link
    - between two processes running on the network
  - associated with a port number and IP addresses
    - port number at transport layer
    - IP address at network layer

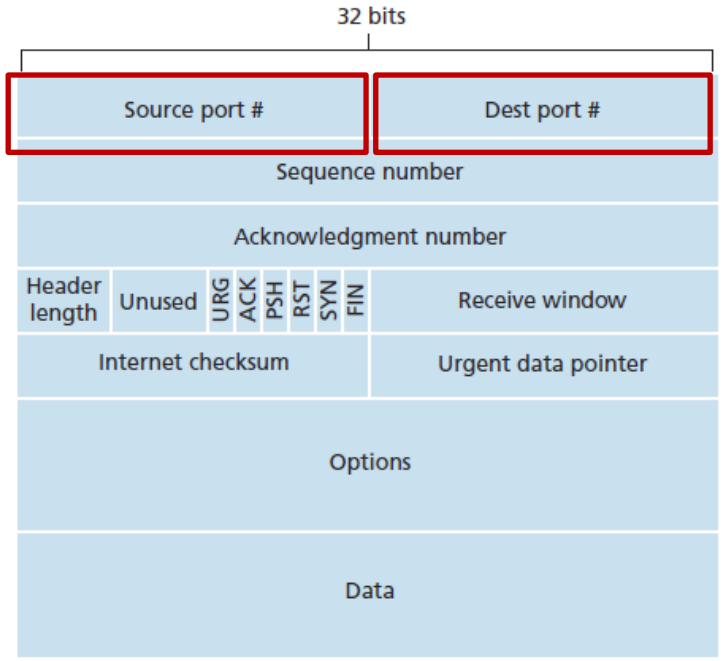


Port Number and IP Address

- Port Number at transport layer
  - Each application (process)
    - has a port number unique in its end system
  - Port number distinguishes various application protocols
    - each port number is a 16-bit number, 0 ~ 2^{16}-1 (65535)



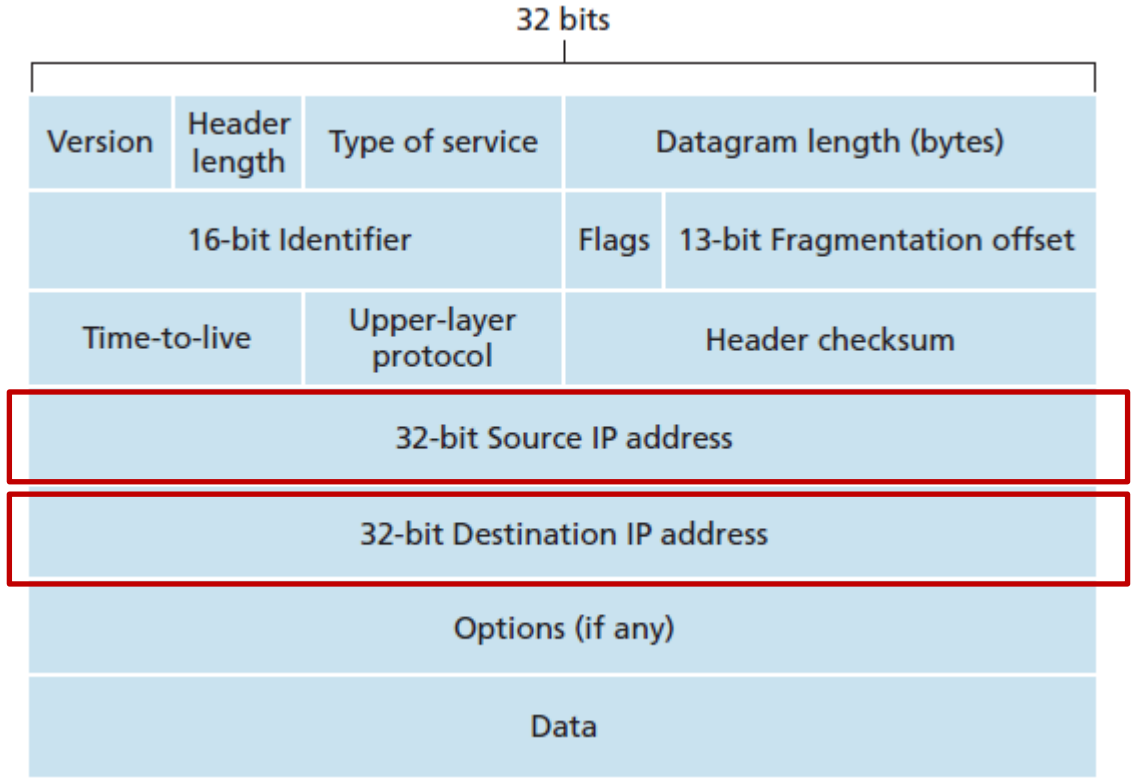
UDP Segment Structure



TCP Segment Structure

Port Number and IP Address

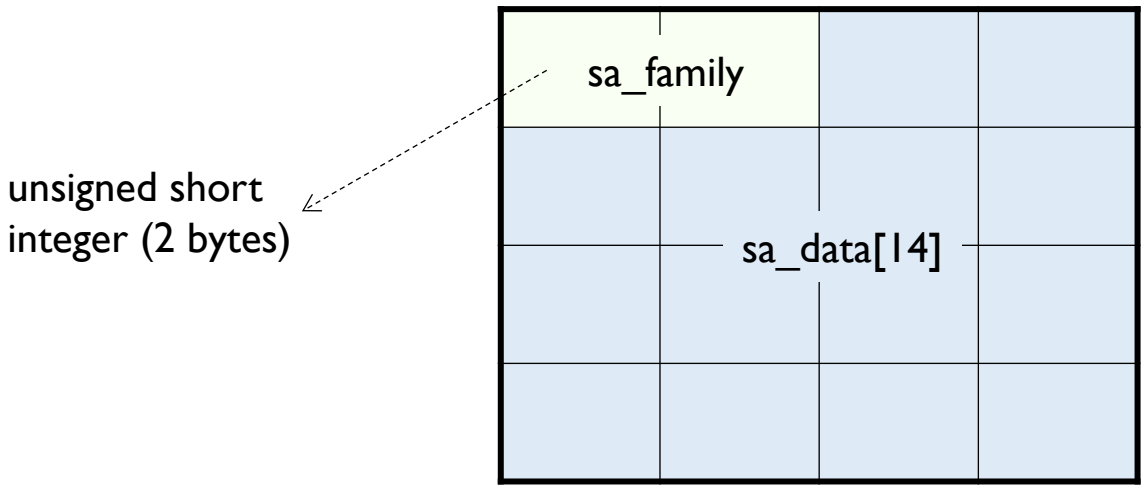
- IP addresses in a network-layer IPv4 packet
  - the datagram plays a central role in the Internet
  - 32-bits (4 bytes)



Generic Socket Address Structure

- Generic socket address structure
  - socket address structure that can support any of protocol families
    - it provides a reference model for address structure

```
#include <sys/socket.h>
struct sockaddr {
        sa_family_t     sa_family;      /* address family: AF_xxx value */
        char            sa_data[14]     /* protocol-specific address */
};
```



IPv4 Socket Addresses

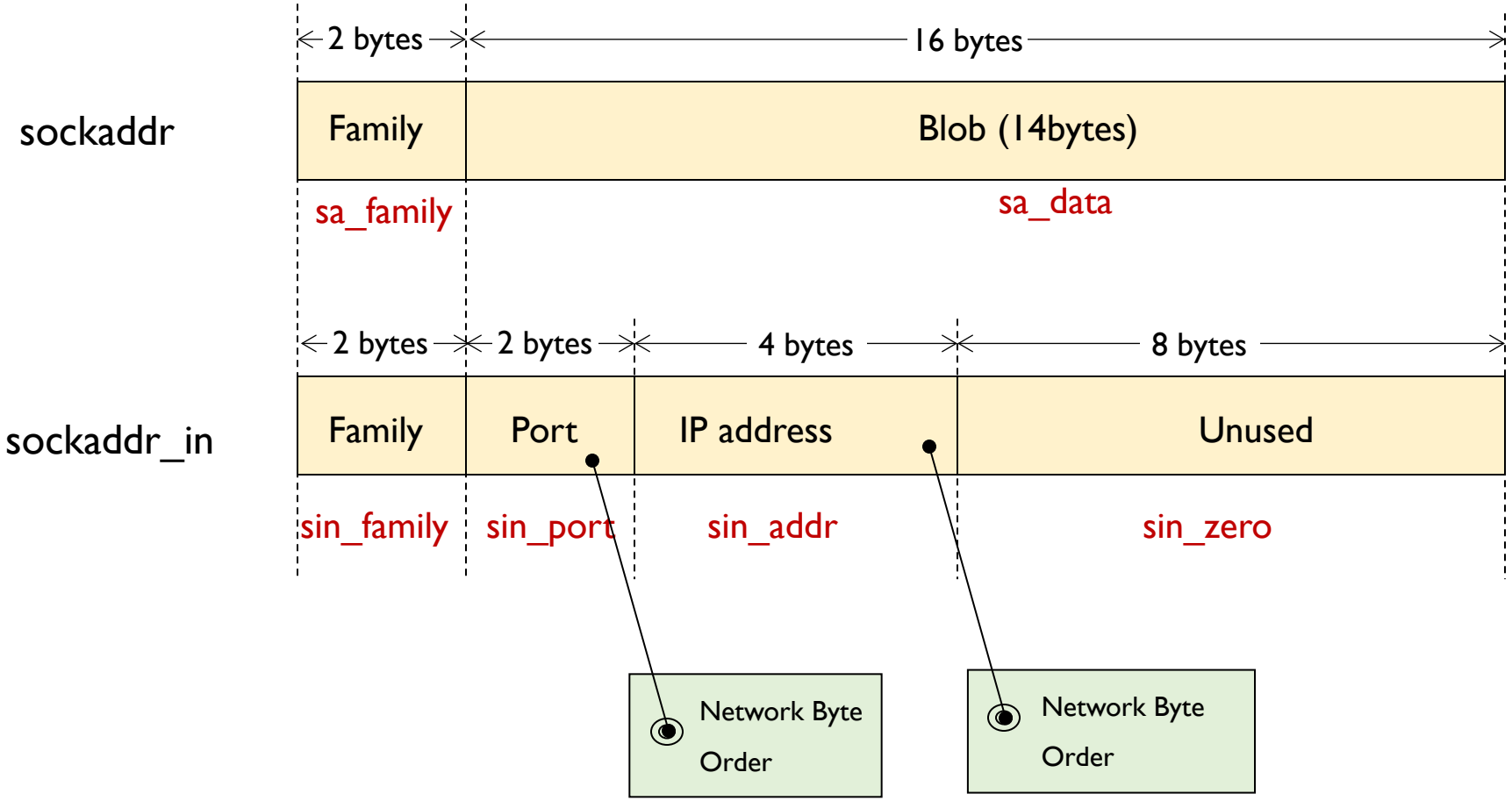
- IPv4 socket address structure
  - defined by AF_INET
  - sockaddr_in : socket address structure to support IPv4 addresses

```
#include <netinet/in.h>
struct sockaddr_in {                    byte
        sa_family_t     2   sin_family;     /* address family */
        uint16_t        2   sin_port;       /* port number */
        struct  in_addr 4   sin_addr;       /* Internet address*/
        unsigned char       sin_zero[8]     /* pad bytes */
};
```

```
struct in_addr {
        unit32_t            s_addr;         /* Internet address (4bytes: 32bits)*/
};
```

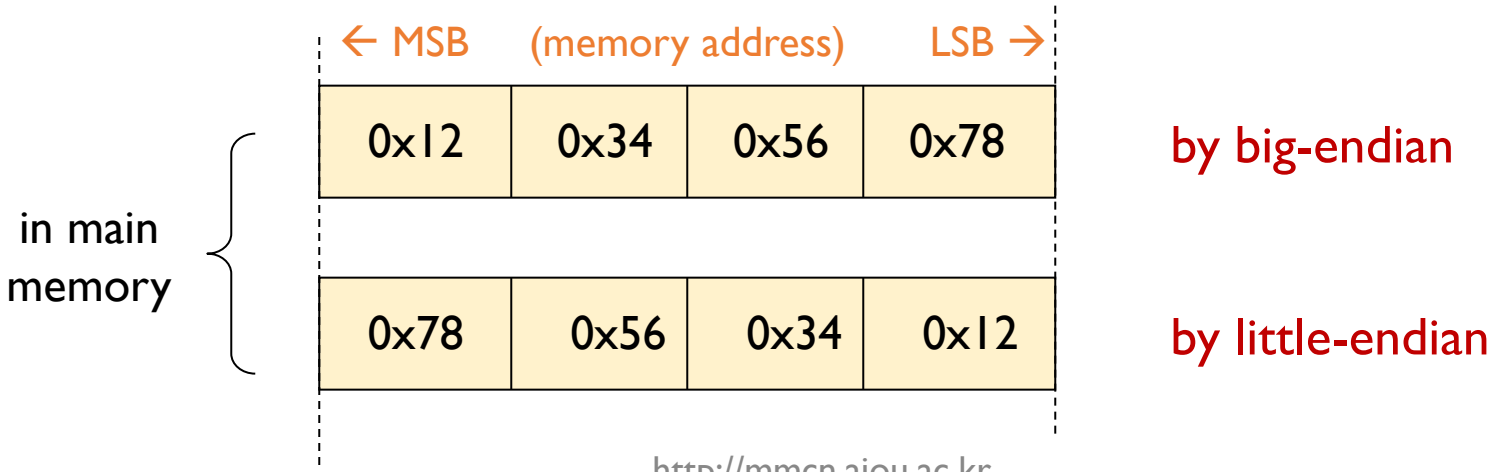
IPv4 Socket Addresses

- Physical layout



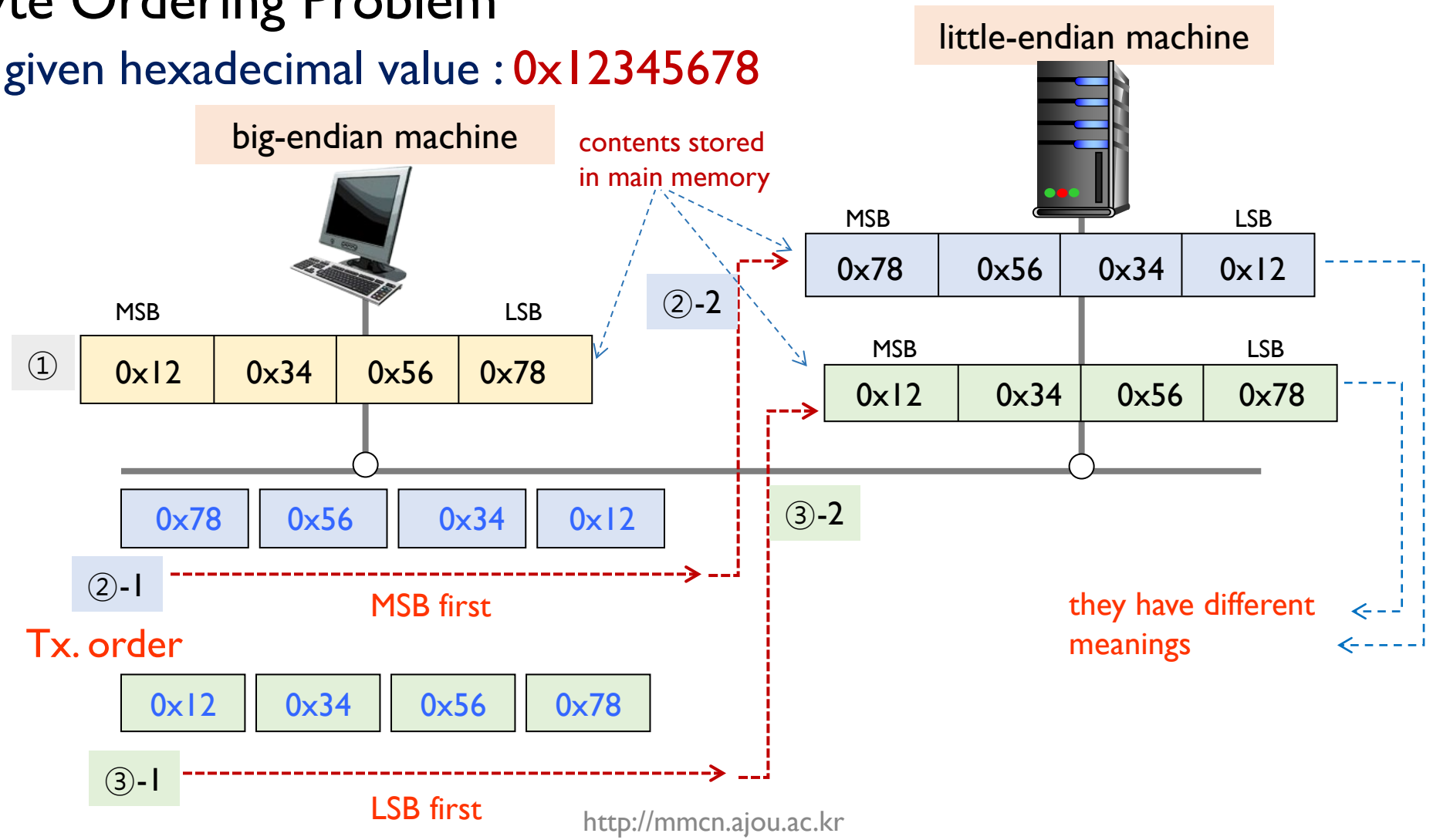
Network Byte Ordering

- Two byte-ordering methods
  - big-endian
    - big-endian machine : Motorola 68000 series
  - little-endian
    - little-endian machine : Intel CPUs
- example
  - given hexadecimal value : 0x12345678



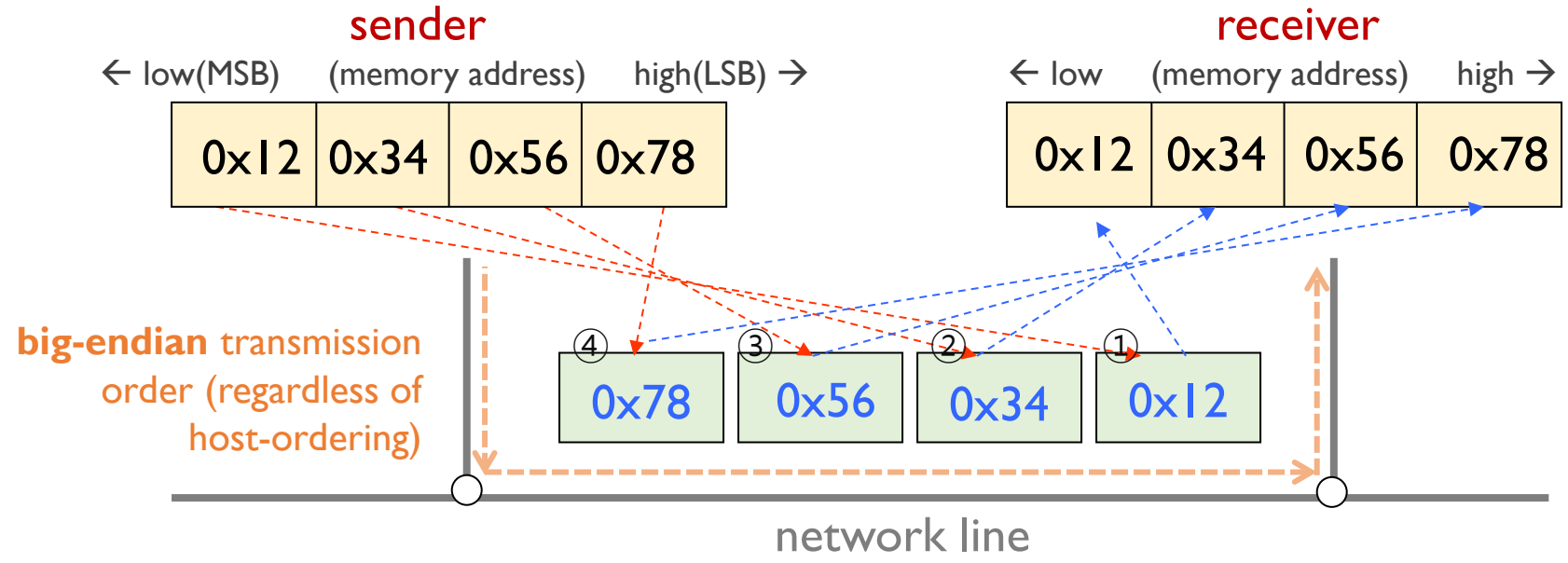
Network Byte Ordering

- Byte Ordering Problem
  - given hexadecimal value : 0x12345678



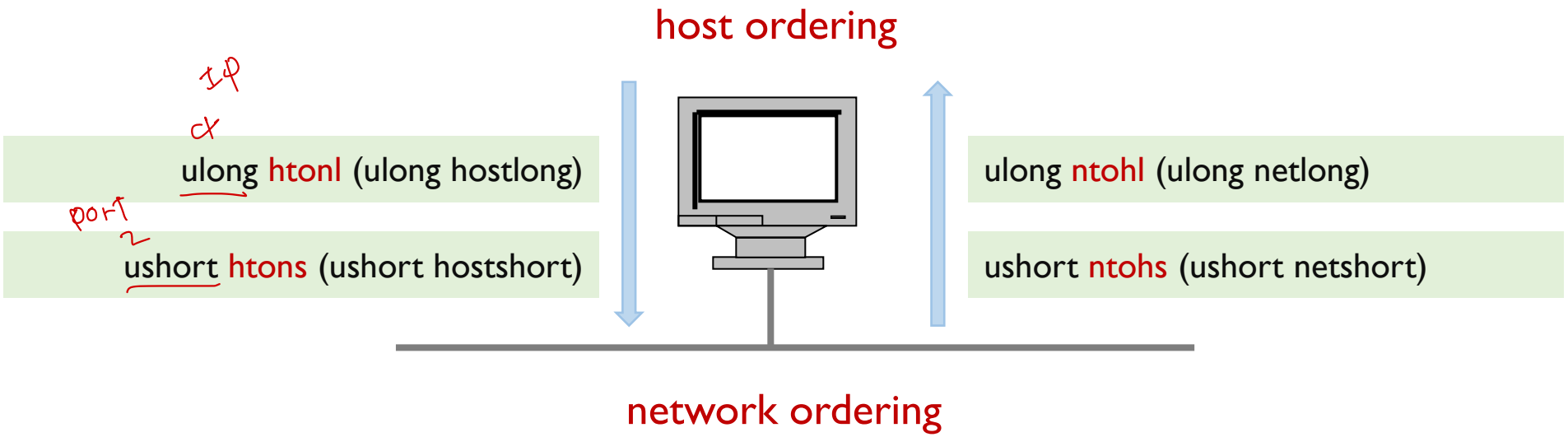
Network Byte Ordering

- Network Byte Ordering
  - big-endian byte ordering is used
    - MSB first
  - example
    - physical memory layout (regardless of host-ordering)



Network Byte Ordering

❖ Endian Conversions

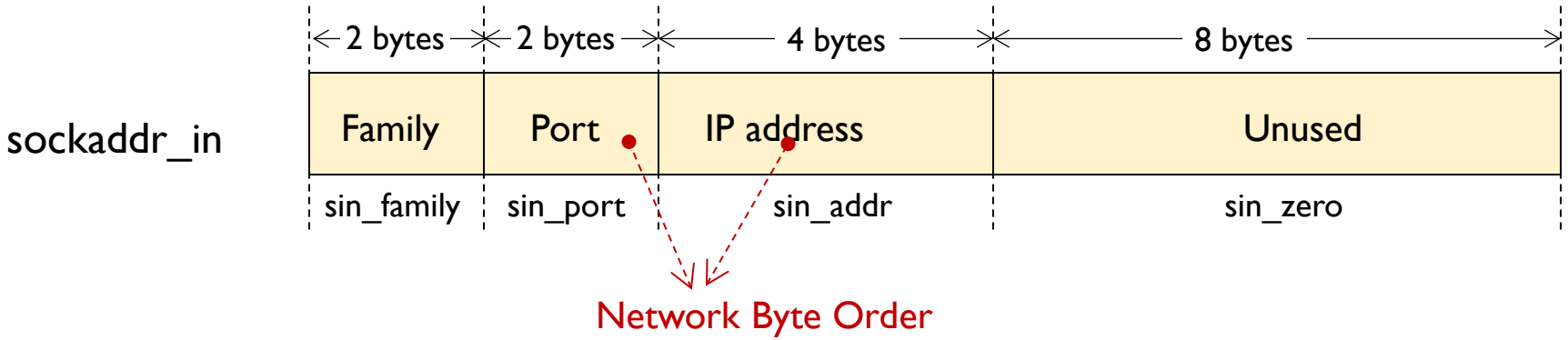


ushort : unsigned short int (16-bit) (e.g. port number)
ulong : unsigned long int (32-bit) (e.g. ip address)

Network Byte Ordering

- sample code
  - Initializing an AF_INET Address

```
1:  struct socketaddr_in echoServAddr;
2:  int adr_len;
3:
4:  memset (&echoServAddr, 0, sizeof echoServAddr);
5:
6:  echoServAddr.sin_family          = AF_INET;
7:  echoServAddr.sin_port            = htons (any_port_number);
8:  echoServAddr.sin_addr.s_addr     = htonl (any_IP_address);
9:  adr_len                          = sizeof echoServAddr;
```



IP Address Manipulation

IP Address (Number) Manipulation

- IPv4 IP address
  - 32-bits (4 bytes) length
  - dotted decimal notation
- IP address (number) manipulation
  - for a given IP address,

192.168.0.123

- how to set the IP address for a socket ?
- how to convert its ordering for networking ?
- how to extract host ID or network ID ?
- and so on….

```
#include <netinet/in.h>
struct sockaddr_in {
        sa_family_t     sin_family;     /* address family */
        uint16_t        sin_port;       /* port number */
        struct  in_addr sin_addr;       /* Internet address*/
        unsigned char   sin_zero[8]     /* pad bytes */
};

struct in_addr {
        unit32_t        s_addr;         /* IP address (32bits)*/
};
```

IP Number Manipulation

- Functions for IP address manipulation
  - headers

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
```

  - functions
    - inet_addr
    - inet_aton
    - inet_ntoa
    - inet_network
    - inet_lnaof
    - inet_netof
    - inet_makeaddr
    - …

inet_addr( )

- function inet_addr( )
  - converts a string of an IPv4 dotted-decimal address into a proper address for IN_ADDR structure (network order) with no return value

```
unsigned long inet_addr (const char *string);
```

return:32bits number with IN_ADDR structure

string of IPv4 dotted decimal notation

  - older function, no longer be used in new code
  - example code

```
struct sockaddr_in addr;                /* AF_INET */
…
addr.sin_addr.s_addr = inet_addr( "127.0.0.95" ) ;
```

uint32_t (network byte order)

string dotted-decimal

inet_aton( )

- function inet_aton( )
  - converts a string of an IPv4 dotted-decimal address into a proper address for IN_ADDR structure (network order) with a return value

```
int inet_aton (const char *string, struct in_addr *addr);
```

return value: non-zero for success, 0 for failure

string of IPv4 dotted decimal address

32bits number with IN_ADDR structure

  - improved version of inet_addr( )
    - it has a return value: non-zero for success, 0 for not-valid address
  - example code

```
struct sockaddr_in addr;                /* AF_INET */
…
int z = inet_aton ("127.0.0.95" , &addr.sin_addr ) ;
```

return value

string dotted-decimal

uint32_t (network byte order)

Discussion

- It seems that inet_addr( ) and inet_aton( ) provide similar function.
- What is the benefit by using inet_aton( ) instead of inet_addr( ) ?

```
struct sockaddr_in addr;                /* AF_INET */
…
addr.sin_addr.s_addr = inet_addr("127.0.0.95" ) ;

// there is no way to check whether inet_addr( ) worked well
```

```
struct sockaddr_in addr;                /* AF_INET */
…
int z = inet_aton ("127.0.0.95" ,  &addr.sin_addr ) ;

// failure operation
if ( z ==0 )
    …
```

inet_ntoa( )

❖ function inet_ntoa( )

- converts an IPv4 address with IN_ADDR structure (network order) into a dotted-decimal ASCII string format

```
char* inet_ntoa ( struct in_addr  addr );
```

return: string of IPv4 dotted decimal address

32bits number with IN_ADDR structure (network order)

- example code

```
struct sockaddr_in addr;                              /* AF_INET */
int z = inet_aton( "127.0.0.23",  &addr.sin_addr ) ;
…
printf( "The IP Address is %s\n ", inet_ntoa( addr.sin_addr ) );
```

- result
  The IP Address is 127.0.0.23

inet_network ( )

- function inet_network( )
  - converts a string of dotted-decimal format into a 32-bts number of IP address in host byte order

```
unsigned long inet_network ( const char *addr );
```

return: 32bits IP address (host ordering)

string of IPv4 dotted decimal address

  - it is convenient when extracting network bits by masking
  - example code

```
unsigned long net_addr;
…
net_addr = inet_network (“192.168.9.1”) & 0xFFFFFF00 ;
```

32bit number of 192.168.9.0 or 0xC0A80900 (host byte order)

return: 32bit IP address (host byte order)

C-class network mask (host byte order)

Discussion

- Do you understand the relationship between
  - inet_network( ) and
  - byte-ordering ?

Etc.

❖ functions inet_lnaof ( ), inet_netof ( ) and inet_makeaddr( )

```
unsigned long inet_lnaof (struct in_addr addr);
```

return: host part of 32 bits IP address (host ordering)

32bits number with IN_ADDR structure (network order)

```
unsigned long inet_netof (struct in_addr addr);
```

return: network part of 32 bits IP address (host ordering)

32bits number with IN_ADDR structure (network order)

```
struct in_addr inet_makeaddr (int net,  int host);
```

return: 32 bits IP address (hntwork ordering)

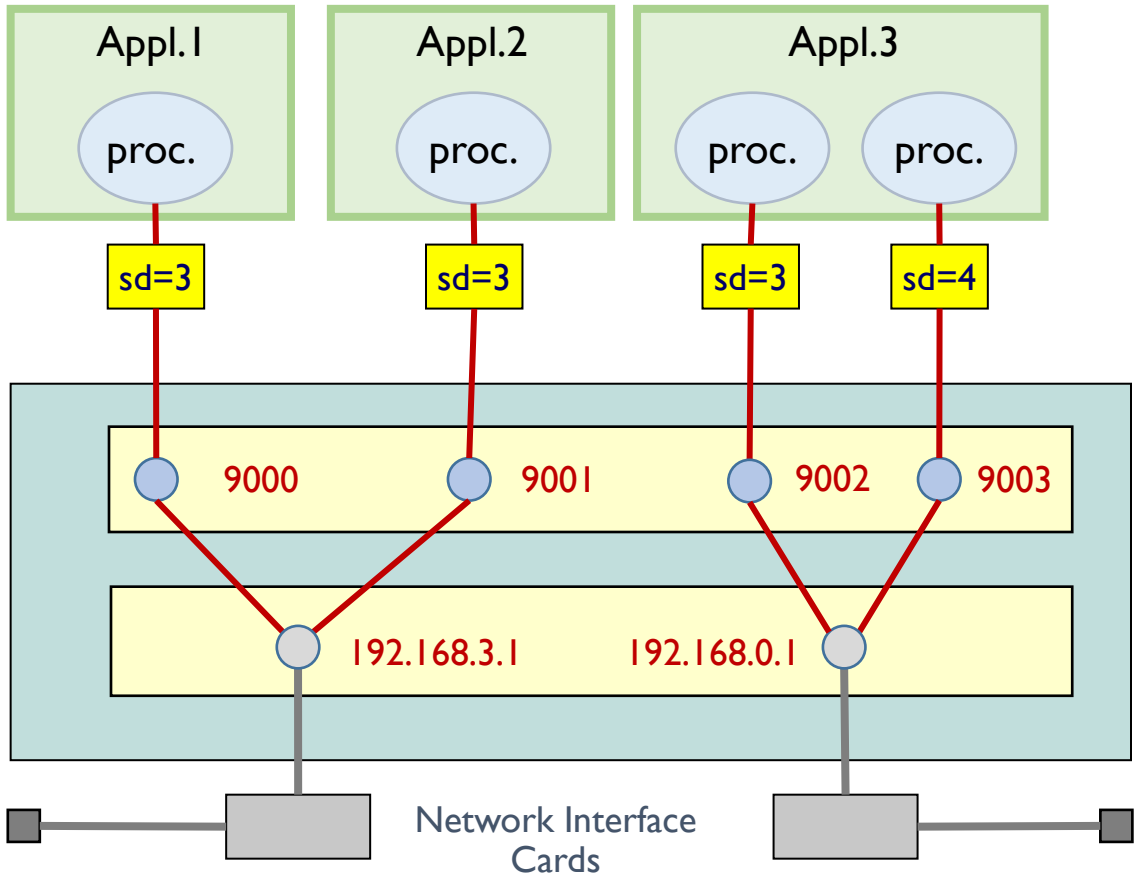
network part 32bits number (host order)

host part 32bits number (host order)

BINDING

What is binding ?

- binding
  - to relate a *socket descriptor* to (*port number* and *IP address*) of the NIC receiving packets



Application Layer

Socket Layer
Socket descriptors

Transport Layer
Port Numbers

IP Layer
IP address

Link Layer
MAC address

bind( ) Function

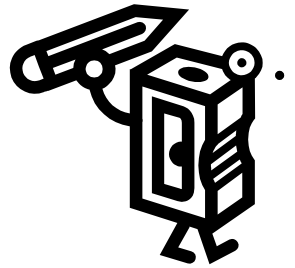
❖ function bind( )

```
#include <sys/types.h>
#include <sys/socket.h>

int bind (int sockfd, struct sockaddr *addr, int addrlen );
```

- arguments
  - sockfd : socket file descriptor created by socket( )
  - addr: address to assign to the socket
  - addrlen : length of the address addr in bytes
- return value
  - 0 (zero) : if successful
  - -1 : otherwise

Discussion

- In the previous lecture note, we learned that struct sockaddr is used only for reference, not for actual use.
- Though, struct sockaddr is one of arguments required for bind( ).
- When we want to use AF_INET and it requires struct sockaddr_in.
- Then how can we use struct sockaddr_in for bind( ) function ?



bind( ) Function

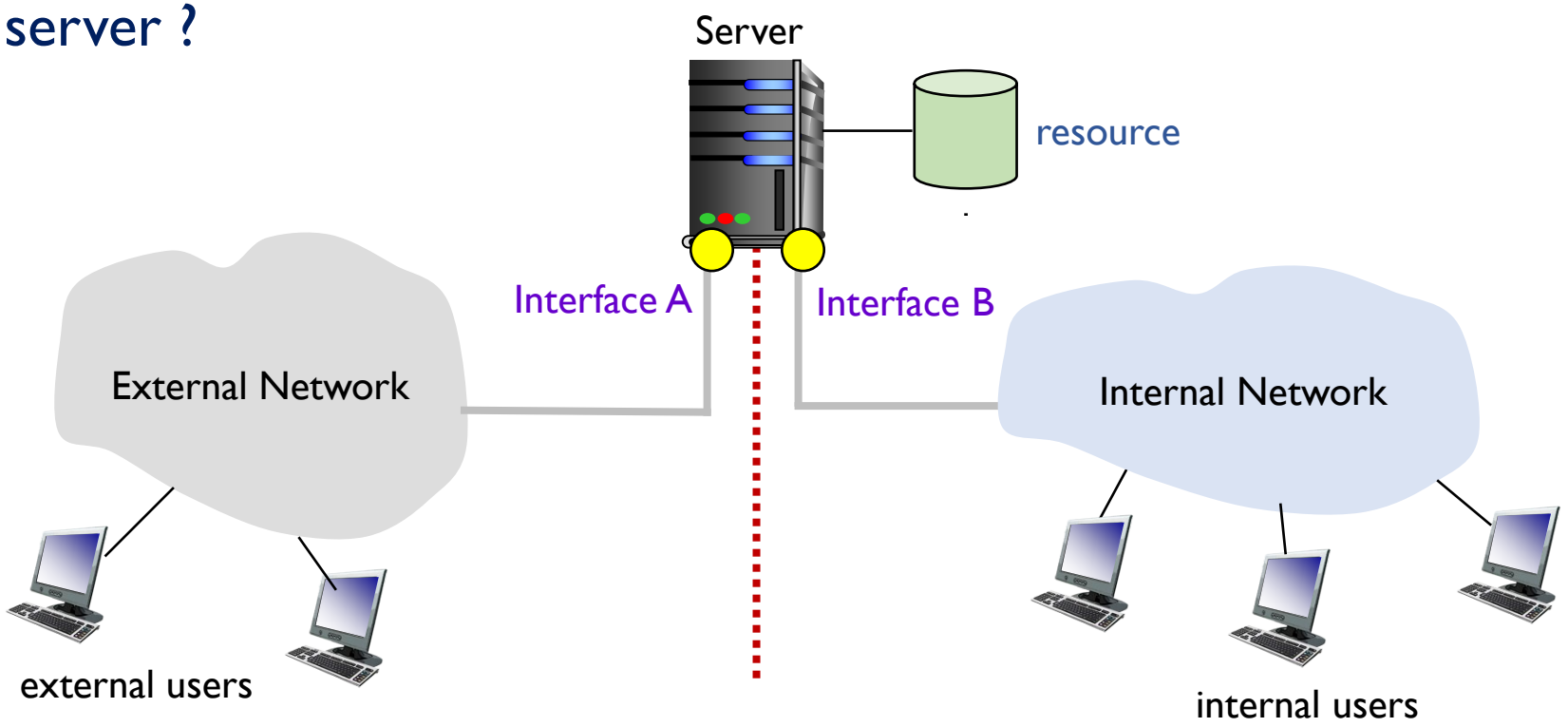
❖ example code

```
int                 z;
int                 sd;
struct sockaddr_in  addr;              /* AF_INET */
int                 addr_len;

/* create a socket */
sd = socket ( AF_INET, SOCK_STREAM, 0 );

/* initialize address */
addr_len = sizeof addr;
memset ( &addr,  0, addr_len );

/* establish address */
addr.sin_family             = AF_INET;
addr.sin_port               = htons (9000);
addr.sin_addr.s_addr        = inet_addr ("192.168.0.1");

/* bind the address to the scoekt */
z = bind ( sd, (struct sockaddr *)&addr,  addr_len );
```

must be network-byte ordered

bind( ) – INADDR_ANY

- bind
  - associates a socket with an address (or interface) to accept packets from other hosts
  - problem : how to allow only the internal users to access to the resource on the server ?



bind( ) – INADDR_ANY

- Binding a socket
  - to a specific interface address
    - set the interface address to the address member of struct sockaddr_in
    - example

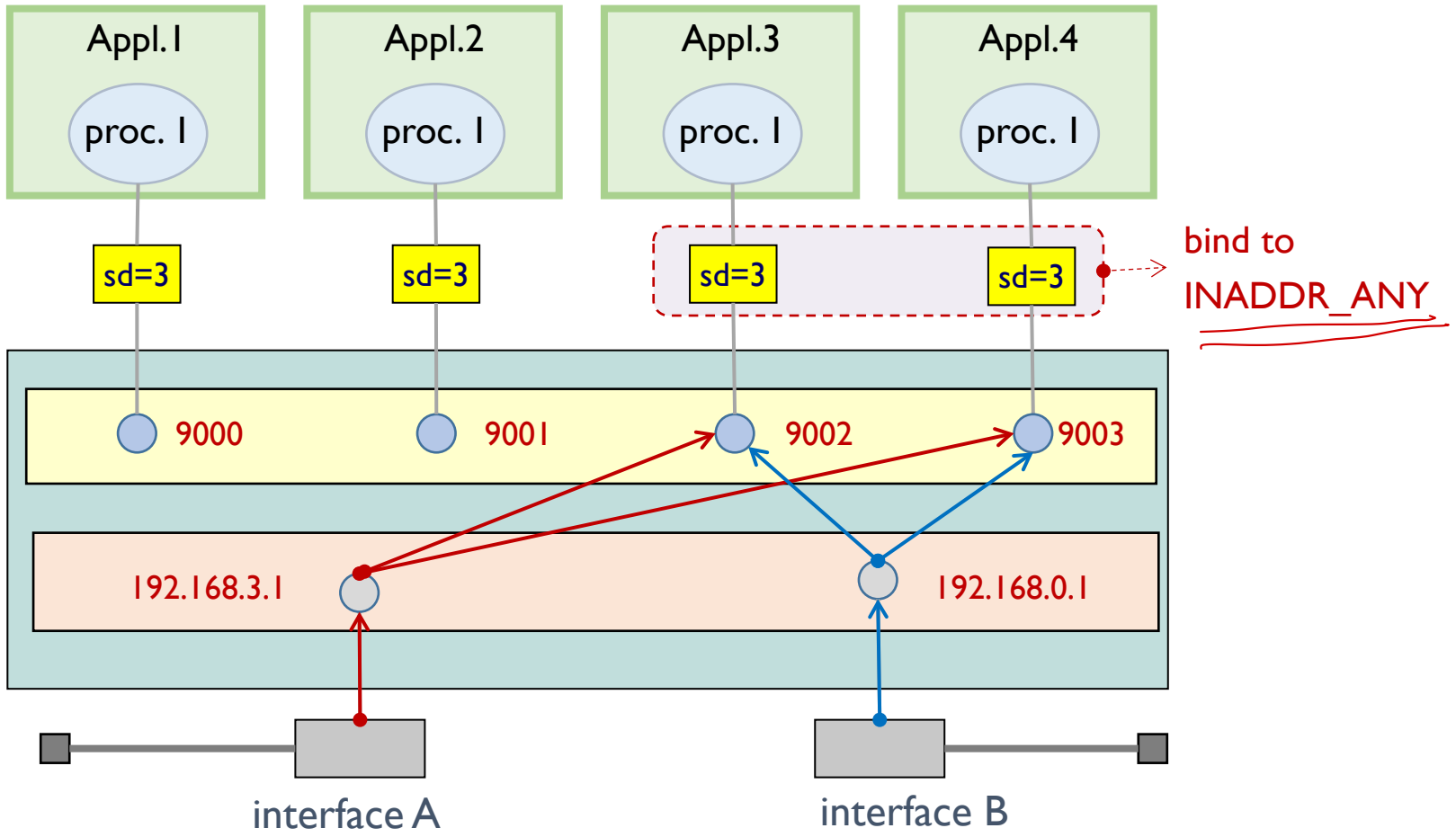
```
addr.sin_addr.s_addr = inet_addr( "192.168.0.1" );
bind ( s, (struct sockaddr *)addr, addr_len);
```

  - to any interface
    - set INADDR_ANY to the address member of struct sockaddr_in
    - example

```
addr.sin_addr.s_addr = htonl ( INADDR_ANY );
bind ( s, (struct sockaddr *)addr, addr_len);
```

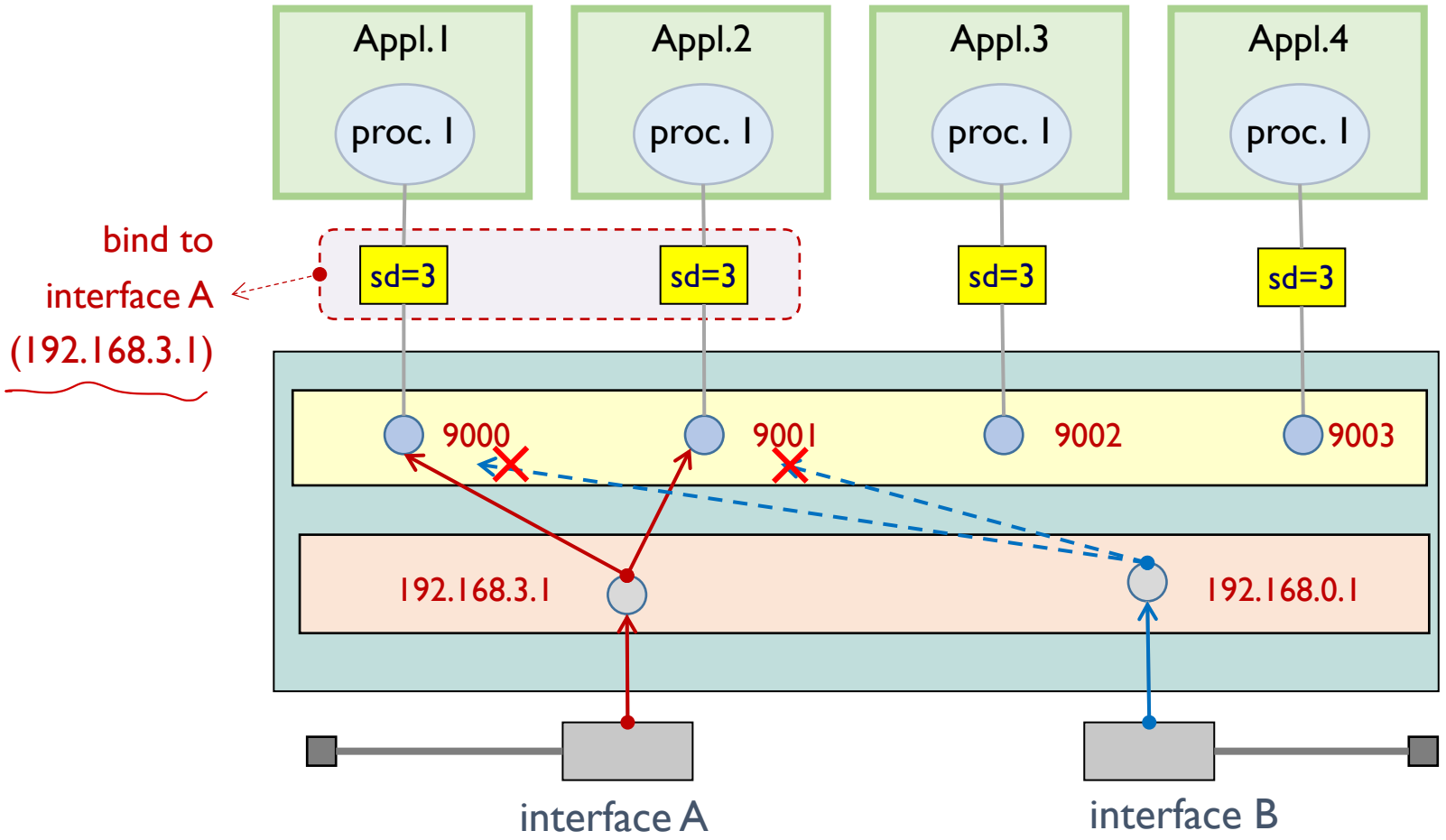
bind( ) – INADDR_ANY

❖ bind with INADDR_ANY



bind( ) – INADDR_ANY

❖ bind with a specific interface A



Questions ?

MR-IoT/AI Convergence

Future Internet & 5G/6G

Intelligent Networking with AI

MMcN

Mobile **Multimedia Convergence** Network Lab.

Homepage: http://mmcn.ajou.ac.kr

facebook: http://www.facebook.com/#!/groups/190702010947314

BK21 FOUR

AI융합네트워크학과